SURVEY

LLMs in Software Security: A Survey of Vulnerability Detection Techniques and Insights

ZE SHENG, Texas A&M University, College Station, TX, United States

ZHICHENG CHEN, Texas A&M University, College Station, TX, United States

SHUNING GU, Texas A&M University, College Station, TX, United States

HEQING HUANG, City University of Hong Kong, Hong Kong, Hong Kong

GUOFEI GU, Texas A&M University, College Station, TX, United States

JEFF HUANG, Texas A&M University, College Station, TX, United States

Published: 21 November 2025
Online AM: 23 September 2025
Accepted: 15 September 2025
Revised: 25 August 2025
Received: 12 February 2025

LLMs in Software Security: A Survey of Vulnerability Detection Techniques and Insights

ZE SHENG, Computer Science and Engineering, Texas A&M University, College Station, USA
ZHICHENG CHEN, Computer Science and Engineering, Texas A&M University, College Station, USA
SHUNING GU, Computer Science and Engineering, Texas A&M University, College Station, USA
HEQING HUANG, Computer Science, City University of Hong Kong, Hong Kong, Hong Kong
GUOFEI GU, Computer Science and Engineering, Texas A&M University, College Station, USA
JEFF HUANG, Computer Science and Engineering, Texas A&M University, College Station, USA

Large Language Models (LLMs) are emerging as transformative tools for software vulnerability detection. Traditional methods, including static and dynamic analysis, face limitations in efficiency, false-positive rates, and scalability with modern software complexity. Through code structure analysis, pattern identification, and repair suggestion generation, LLMs demonstrate a novel approach to vulnerability mitigation.

This survey examines LLMs in vulnerability detection, analyzing problem formulation, model selection, application methodologies, datasets, and evaluation metrics. We investigate current research challenges, emphasizing cross-language detection, multimodal integration, and repository-level analysis. Based on our findings, we propose solutions addressing dataset scalability, model interpretability, and low-resource scenarios.

Our contributions include: (1) a systematic analysis of LLM applications in vulnerability detection; (2) a unified framework examining patterns and variations across studies; and (3) identification of key challenges and research directions. This work advances the understanding of LLM-based vulnerability detection. The latest findings are maintained at https://github.com/OwenSanzas/LLM-For-Vulnerability-Detection

CCS Concepts: • **Security and privacy → Software security engineering;**

Additional Key Words and Phrases: Large language models, vulnerability detection, cybersecurity

ACM Reference Format:
Ze Sheng, Zhicheng Chen, Shuning Gu, Heqing Huang, Guofei Gu, and Jeff Huang. 2025. LLMs in Software Security: A Survey of Vulnerability Detection Techniques and Insights. *ACM Comput. Surv.* 58, 5, Article 134 (November 2025), 35 pages. https://doi.org/10.1145/3769082

1 Introduction

Vulnerability detection plays an important part in the design and maintenance of modern software. Statistical evidence indicates that approximately 70% of security vulnerabilities originate from defects in the software development process [4]. According to the metrics provided by **Common**

Authors' Contact Information: Ze Sheng (corresponding author), Computer Science and Engineering, Texas A&M University, College Station, Texas, USA; e-mail: zesheng@tamu.edu; Zhicheng Chen, Computer Science and Engineering, Texas A&M University, College Station, USA; e-mail: zhicheng@tamu.edu; Shuning Gu, Computer Science and Engineering, Texas A&M University, College Station, Texas, USA; e-mail: shuning@tamu.edu; Heqing Huang, Computer Science, City University of Hong Kong, Hong Kong, Hong Kong; e-mail: heqhuang@cityu.edu.hk; Guofei Gu, Computer Science and Engineering, Texas A&M University, College Station, Texas, USA; e-mail: guofei@tamu.edu; Jeff Huang, Computer Science and Engineering, Texas A&M University, College Station, Texas, USA; e-mail: jeffhuang@tamu.edu.

ACM 0360-0300/2025/11-ART134
https://doi.org/10.1145/3769082

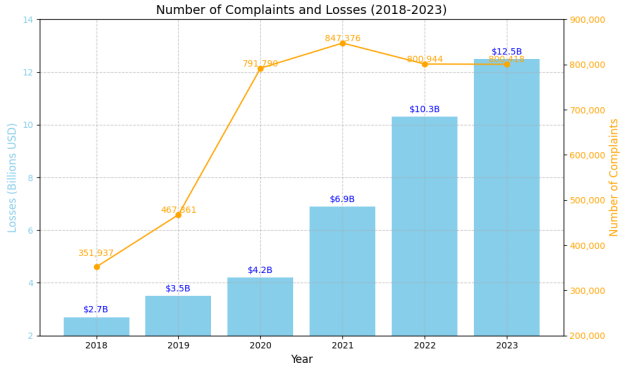

Fig. 1. Complaints and financial losses from 2018–2023.

Vulnerabilities and Exposures Numbering Authorities (**CNAs**), a growth is witnessed that in the past 5 years, about 120,000 CVEs have been discovered and reported [22]. According to FBI's cybercrime report shown in Figure 1, the period from 2018 to 2023 suffers from a large amount of cybersecurity crimes and complaints. A recent example is the CrowdStrike incident in July 2024 [126], where a faulty software update caused widespread system crashes across critical infrastructure sectors including healthcare, transportation, and finance. Therefore, enhanced focus and investment in vulnerability detection technology is in demand.

State-of-the-art vulnerability detection approaches/tools can be broadly classified into static analysis and dynamic analysis [20, 88, 114, 150]. Static analysis examines source code or bytecode to identify potential security vulnerabilities while dynamic analysis does it during program execution, including techniques such as fuzz testing [117]. Fuzz testing identifies potential vulnerabilities by inputting random or specific invalid data into applications and observing system responses. However, with the increasing scale of software systems, both traditional static and dynamic analysis approaches have distinct limitations. For example, static and dynamic analysis tools suffer from high false positive rates, low efficiency and vast effort for overwhelmed amount of types of vulnerabilities [64, 128].

Large Language Models (**LLMs**), as an advancement in **natural language processing** (**NLP**), are trained by deep learning techniques, particularly the Transformer architecture to focus on diverse NLP tasks [127]. Recently, LLMs have shown remarkable abilities in software development area [60]. Based on these capabilities, several LLM-driven vulnerability detection methodologies and tools have been proposed by researchers. This new trend has attracted attention from both cybersecurity experts and machine learning researchers, potentially revolutionizing this field. For example, in 2024 DARPA held the **Artificial Intelligence Cyber Challenge** (**AIxCC**), where competitors only leverage general-purpose LLMs (GPT series, Claude series, and Gemini series) for vulnerability detection, reproduction, and patching [25]. The competition brought together leading experts in both cybersecurity and machine learning all over the world, indicating the significant potential of applying LLMs to vulnerability detection.

The increasing adoption of LLMs in vulnerability detection is evident from initiatives like AIxCC, showcasing the rapid evolution of their capabilities and the diversity of approaches in this field. Despite growing interest and significant research efforts, a systematic survey that thoroughly examines LLM-based detection methods are still lacking.

This article addresses this gap by presenting the first comprehensive survey focused on understanding the strengths and weaknesses of LLMs in vulnerability detection. To provide a

structured and holistic analysis, we construct our survey around four key aspects: (1) identifying effective LLM architectures for security tasks, (2) evaluating benchmarks, datasets, and metrics for reliable assessment, (3) analyzing techniques to uncover best practices, and (4) recognizing challenges to guide future research directions. These aspects collectively highlight the current state of the field and its potential for advancement.

Based on these aspects, we focus on the following **research questions** (**RQs**):

— **RQ1**. What LLMs have been applied to vulnerability detection?
— **RQ2**. What benchmarks, dataset and metrics have been designed to evaluate vulnerability detection?
— **RQ3**. What techniques have been used in LLM for vulnerability detection?
— **RQ4**. What are the challenges that LLMs are facing in detecting vulnerabilities and potential directions to solve them?

To analyze and summarize the RQs, we selected more than 80 papers (with about 60 highly related papers) from over 500 papers to ensure the recency (2019–2024) and the relevance (focusing on LLMs in vulnerability detection). The general structure of RQs is shown in Figure 5. Therefore, this survey will not discuss traditional machine learning approaches (conventional CNN, RNN and LSTM) in vulnerability detection, which were predominantly used between 2014 and 2020.

In general, our key findings can be summarized as the positive and key gaps:

- **The Positive:** Cybersecurity community has experienced a positive impact from LLM, evidenced by a substantial increase in published articles in recent years. The contributions span multiple areas, including vulnerability localization, detection and analysis. C, Java and Solidity have emerged as predominant focuses in this domain. Research methodologies consistently emphasize three key components: LLM implementation, prompt engineering, and semantic processing methods. Moreover, multi-agent approaches are widely used because of the decomposition of complex vulnerability detection challenges into manageable sub-problems.
- **Key Gaps:**
  - **Narrow Scope and Limited Repository-Level Coverage:** Current work often restricts itself to binary classification of function-level vulnerabilities within small, specialized datasets. Moreover, few studies address the detection and reproduction of vulnerabilities at the repository level, where cross-file dependencies and longer call stacks pose significant challenges.
  - **Rapid Advances in Frontier LLMs:** Breakthroughs in 2023–2024 indicate that fine-tuning and leveraging frontier LLMs will be critical for future progress, yet most research to date relies on less-capable traditional models.
  - **Insufficient Context Awareness:** There is insufficient attention to complex, multi-file dependencies and long call stacks. Although neuro-Symbolic approaches (e.g., CodeQL, Bear with LLMs) have shown promise on large-scale projects, improved taint propagation modeling, more efficient LLM reasoning, and cross-language adaptation are still required.
  - **Vulnerability Type Imbalance:** Memory-related vulnerabilities (e.g., buffer overflow issues) receive disproportionately higher detection accuracy, while logical vulnerabilities remain relatively underexplored.
  - **Dataset Limitations:** Existing datasets are narrowly scoped and have a data leakage problem. A dedicated and comprehensive data set specifically tailored for LLM-based vulnerability detection is urgently needed to drive both fundamental and applied research.

The following sections present a comprehensive analysis of LLMs in vulnerability detection. Given the extensive scope of this survey, this section outlines the structure, main themes, and narrative flow of our analysis. A visualization of this paper's structure is shown in Figure 2.

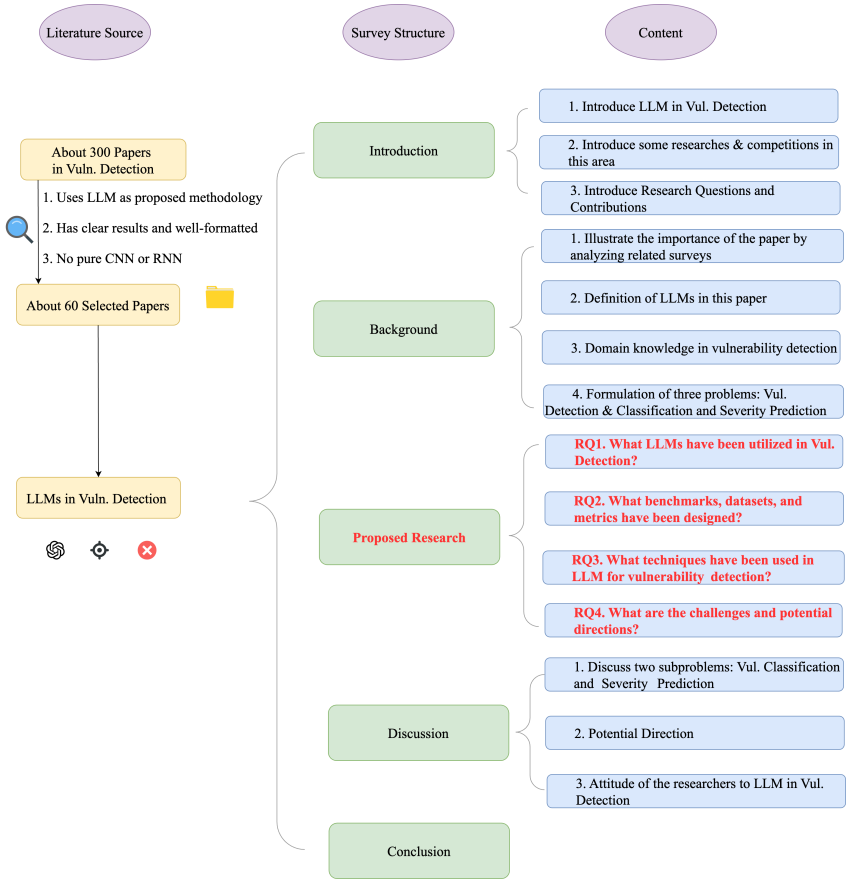

Fig. 2. Survey structure and research selection.

2 Background

2.1 Paper Selection and Scope

To ensure a comprehensive and systematic review, we initiated our search with leading security conferences, including the IEEE Symposium on **Security and Privacy** (**S&P**), USENIX Security, and the ACM Conference on **Computer and Communications Security** (**CCS**). It is also noteworthy that several studies appeared in software engineering venues, such as the **International Conference on Software Engineering** (**ICSE**) and IEEE Transactions on Software Engineering, reflecting the close connection between vulnerability detection and software development. Our supplemental file summarizes a selection of the representative papers considered in this survey.

Then we searched by extracting key terms such as "vulnerability detection," "LLM," "large language model," and "AI" from papers published in conferences and journals. Using these keywords, we conducted iterative searches every three weeks, refining the selection over time. Over a two-month period, we screened approximately 500–600 papers and selected about 60 highly relevant studies.

In recent years, we observed a clear increase in the number of publications in this field. To highlight the timeliness of this survey, we focus on reviewing the most recent contributions. Figure 3 illustrates the distribution of the reviewed papers by publication year between 2017 and

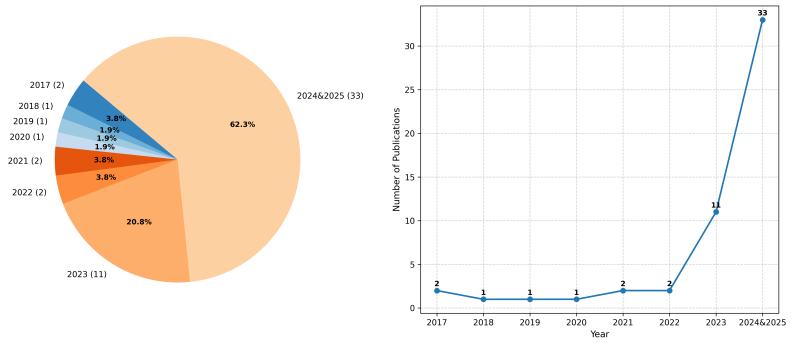

Fig. 3. Distribution of selected papers on vulnerability detection by publication year (2017–2025).

2025. As shown, the years 2023, 2024, and 2025 mark a rapid growth phase in this research area, indicating that our survey can serve as a timely and valuable roadmap for future studies.

This survey focuses exclusively on the application of LLMs in vulnerability detection, analyzing techniques, datasets, benchmarks, and challenges. We reviewed works targeting programming languages like C/C++, Java, and Solidity, which are the primary focus areas for LLM-based vulnerability detection. Studies centered on traditional machine learning methods, such as CNNs and RNNs, and those unrelated to vulnerability detection, such as malware analysis or network intrusion detection, were excluded.

In terms of datasets, we primarily evaluated function-level and file-level granularity, noting that C/C++ datasets dominate the field. However, repository-level datasets that better reflect real-world development scenarios are significantly lacking. This limitation poses challenges for LLMs in generalizing to complex, multi-file vulnerabilities.

By clearly defining the scope and adopting a rigorous, keyword-driven selection process, we aim at ensuring the robustness and relevance of this survey while laying a solid foundation for future research.

2.2 Related Reviews

In the past five years, many studies have been proposed on leveraging LLMs for vulnerability detection. Several comprehensive surveys have been presented on vulnerability detection techniques, covering traditional approaches (static/dynamic analysis) and machine learning methods (CNN, RNN) [4, 16, 32, 151, 162]. They do not specifically address the integration of LLMs in vulnerability detection. Yao et al. [146] reviewed LLMs in the S&P domain, proposing their positive impacts, potential threats, and inherent threat. However, their analysis focuses on a broad overview of these issues rather than providing a methodological summary of LLM-based detection approaches. Xu et al. [141] presented an overview of LLMs in the entire cybersecutiry domain, including malware analysis, network intrusion detection, and so on. Our survey specifically focuses on LLM-based vulnerability detection with a more detailed summary of techniques and methodologies. At the same time, Zhou et al. [159] investigated how LLMs are adapted for vulnerability detection and repair. While their work provides valuable insights, our survey differs in several aspects: (1) by the time of writing, both OpenAI and Anthropic have released more powerful LLMs (GPT-4o, o1 and Claude 3.5 Sonnet) that have stronger inference abilities and larger context windows; (2) we conduct a comprehensive analysis of benchmarks and evaluation metrics for LLM-based vulnerability detection systems; (3) we focus more on the details of vulnerability detection and understanding.

2.3 Large Language Models (LLMs)

LLMs have emerged as a significant progress in the evolution of language models [155]. The Transformer architecture has enabled unprecedented scaling capabilities. LLMs are characterized by their massive scale, typically incorporating hundreds of billions of parameters trained on vast corpus. Therefore, it leads to remarkable capabilities in general human tasks [23].

2.4 Vulnerability Detection Problem

2.4.1 Domain Knowledge. Some popular vulnerability databases, such as **Common Weakness Enumeration (CWE)**,[1] **Common Vulnerabilities and Exposures (CVE)**,[2] **Common Vulnerability Scoring System (CVSS)**[3] and **National Vulnerability Database (NVD)**,[4] have been built to record the definition and evaluation of common vulnerabilities. CWE focuses on all vulnerabilities in the software development lifecycle (from development to maintenance.) Rather than focusing on specific real-world security vulnerabilities (e.g., Heartbleed and Log4Shell), CWE focuses on the root causes of these real-world vulnerabilities like Use-After-Free (CWE-416) and Out-of-bounds Write (CWE-787). CVE is a public community that identifies and catalogs security vulnerabilities in software and hardware. The community will allocate a unique identifier to each real-world vulnerability. For example, the identifier of Log4Shell is CVE-2021-44228. CVSS is a standardized framework for assessing the risk level of a vulnerability through a number of metrics: exploitability, impact, exploit code maturity, and remediation level, and so on. These metrics result in an overall score on a scale of 0 to 10, and severity from low to critical. Log4Shell was given a CVSS score of 10 (severity: critical). NVD is a database that contains basic information about real-world vulnerabilities, such as CVE identifier, technical details of the vulnerability, CVSS score, and mitigation recommendations. For instance, NVD records that Log4Shell can be exploited to remotely execute code on the victim server by constructing malicious JNDI statements, which is caused by Improper Input Validation (CWE-20) and Uncontrolled Resource Consumption (CWE-400).

2.4.2 Vulnerability Detection. Vulnerability detection serves as the core focus of all selected papers in this survey, representing the primary application of LLMs in software security. The fundamental task can be formally defined as a binary classification problem:

Let C_i denote the input source code and VD_i represent an LLM-driven vulnerability detector. The output $Y_i \in \{0, 1\}$ indicates the vulnerability status where $Y_i = 1$ indicates that the code is vulnerable, and $Y_i = 0$ indicates that the code is non-vulnerable.

Figure 4 shows an example workflow of vulnerability detection in most of selected researches, and two subproblems: vulnerability classification and severity prediction.

2.4.3 Vulnerability Classification. Beyond binary detection, some studies explore LLMs' capability in multi-class vulnerability classification to enhance model reliability. This task requires LLMs to not only identify the presence of vulnerabilities but also determine their specific types according to established standards like CWE.

Formally, vulnerability classification can be defined as: Let C_i denote the input source code and VC_i represent an LLM-driven vulnerability classifier. The output Y_i indicates the specific vulnerability type: $Y_i = VC_i(C_i) \in \{type_1, type_2, ..., type_n\}$, where $type_i$ could be vulnerability names (e.g., Buffer Overflow, SQL Injection) or standardized identifiers (e.g., CWE-119, CWE-89).

[1] https://cwe.mitre.org/
[2] https://www.cve.org/
[3] https://www.first.org/cvss/
[4] https://nvd.nist.gov/

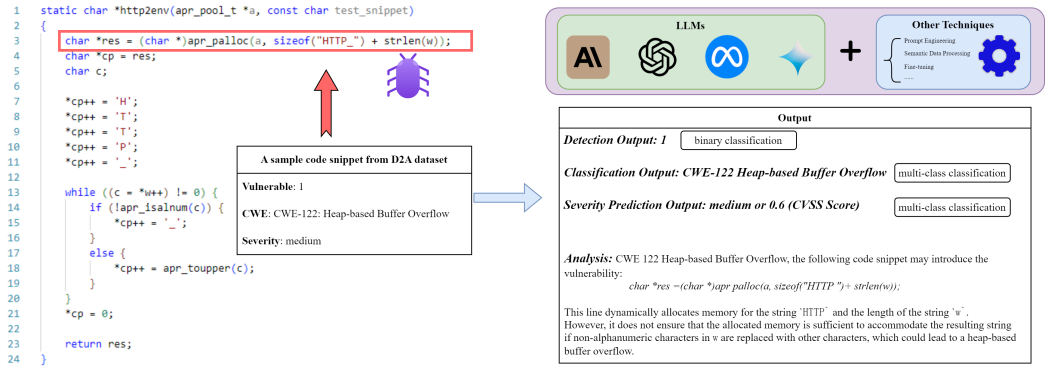

Fig. 4. An example workflow of Vuln. Detection, classification and severity prediction.

For example, when examining a code snippet, an LLM might not only detect its vulnerability but also classify it as "CWE-79: Cross-site Scripting (XSS)", providing more detailed guidance for security mitigation.

2.4.4 Vulnerability Severity Prediction. Some studies extend vulnerability analysis to include severity prediction alongside detection. This can be formulated as either a multi-class classification problem or a regression task, depending on the granularity of severity measurement. Formally, let C_i denote the input source code and VS_i represent an LLM-driven severity predictor. The output Y_i can be defined in two forms: it may represent a severity score such as "low," "medium," or "high," based on the vulnerability severity level of the input source code C_i.

Different studies have adopted varying approaches to severity prediction:

— Categorical Classification: Alam et al. [2] employ a three-level classification system (high, medium, low) for straightforward severity assessment.
— Score Prediction: Fu et al.[41] requires LLMs to predict numerical CVSS scores on a scale of 0 to 10, providing more precise severity measurements.

This additional severity information helps prioritize vulnerability remediation efforts and allocate security resources more effectively in practical applications.

2.5 Ethical Considerations and Potential Misuse

In addition to methodological advances, it is essential to consider the ethical implications and potential misuse of LLM-based tools for vulnerability detection. On the one hand, such models offer defenders unprecedented capabilities to identify and remediate flaws. On the other hand, the same techniques can be exploited by adversaries to accelerate exploit generation, lower the barrier to entry for attackers, and enable large-scale automated vulnerability discovery [36, 67]. This dual-use concern has been broadly recognized in the AI security community [19, 141, 146], yet it has not been systematically examined in the context of vulnerability detection surveys.

A further ethical challenge arises from data privacy and intellectual property. Because LLMs are trained on large-scale code repositories, they may inadvertently memorize and regenerate sensitive or proprietary fragments, raising issues of data handling, licensing compliance, and potential leakage of confidential information [19, 55, 62]. Moreover, existing evaluations of LLM-based vulnerability detection predominantly emphasize accuracy and efficiency, while rarely considering downstream societal or organizational impacts, such as how model outputs or disclosure practices might inadvertently increase risks in real-world environments[6].

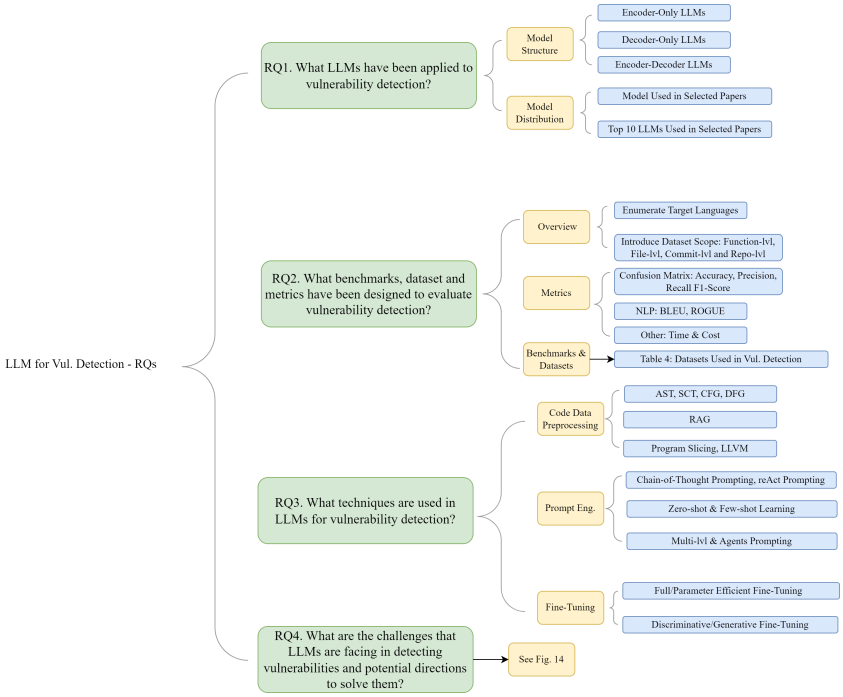

Fig. 5. Research questions.

Although prior reviews in the broader domains of S&P have noted these concerns [42, 100], systematic analysis tailored specifically to LLM-based vulnerability detection remains limited. We argue that future research should integrate responsible deployment considerations alongside technical innovation[116]. Safeguards may include controlled access to high-capability models, alignment with responsible disclosure practices, red-teaming evaluations to assess adversarial misuse, and provenance tracking of training data. By coupling technical progress with ethical safeguards, the community can better ensure that advances in vulnerability detection yield positive security outcomes.

2.6 Real-World Use of LLM in Vulnerability Detection

In real-world applications, the approaches can mainly be categorized into two types: FullScan and DeltaScan [26]. FullScan is responsible for comprehensive vulnerability detection of an entire codebase, such as when preparing a major release or introducing a totally new external library. DeltaScan, on the other hand, focuses on analyzing recent changes to ensure that new changes do not introduce new vulnerabilities [26].

OSSFuzzGen is a mainstream open-source approach for FullScan [49]. In practice, software developers can adopt this approach by first integrating their projects into OSS-Fuzz. Next, LLMs are used to identify coverage gaps and generate new fuzz targets.If build errors occur, they can be automatically repaired through iterative feedback. Finally, the fuzz targets are run continuously to detect crashes. This workflow enables comprehensive FullScan-style detection before major releases, and improving coverage, lowering costs, and accelerating the detection of vulnerabilities across large codebases [49].

Currently, DeltaScan lacks a mature and widely adopted open-source project like OSSFuzzGen, but many security researchers have already started to dive in this area [24]. Thus, there are many available open-source attempts, such as FuzzBrain [104], Buttercup [124], and so on. Whenever a codebase introduces a new commit, the two vulnerability reasoning systems can analyze that commit and then generate a fuzz driver, trying to find a crash. This way, it can prevent newly introduced commits from bringing in new vulnerabilities.

3 Research Results

3.1 Overview

Our analysis shows that research on LLM-based vulnerability detection mainly focuses on C/C++, Java, and Solidity. Each language has unique challenges and research priorities. Studies on C/C++ focus on memory-related vulnerabilities, which are critical in this domain. Java research addresses framework-specific vulnerabilities and complex interactions across components in web applications. Solidity research targets vulnerabilities in smart contracts, which are central to blockchain security.

Recently, large decoder-only models, such as GPT and CodeLlama, have become the main choice due to their size and strong generalization abilities. These models are used in 65% of fine-tuning experiments. For example, Alam et al. [2] fine-tuned GPT-4 and achieved 99% accuracy in Solidity vulnerability detection. Before this, encoder-only models like CodeBERT and GraphCodeBERT were widely used. Advances in prompt engineering have also improved detection performance. Chain-of-Thought prompting is now common for large models and enhances their reasoning abilities for complex code [29].

Most current datasets focus on function-level and file-level vulnerabilities, with C/C++ as the dominant target language. Examples include the Devign [161] and CVEFixes [7] datasets. However, there is a lack of repository-level datasets that better reflect real-world scenarios. This gap limits the practical use of LLMs in vulnerability detection.

Future research should address challenges in cross-file and complex-context vulnerability detection. It should develop better methods for representing code semantics and create realistic repository-level datasets. These steps will improve the applicability and reliability of LLMs in this field.

3.2 RQ1. What LLMs Have been Applied to Vulnerability Detection?

In this article, we use the LLM categorization and taxonomy outlined by Pan et al. [108] and classify the primary LLMs into three architectural groups: (1) encoder-only, (2) encoder-decoder, and (3) decoder-only models. Due to space constraints, we will briefly introduce some representative LLMs in each category. Table 1 gives a clear overview of the strengths and weaknesses of each category of methods based on their inherent capabilities and limitations.

Encoder-only LLMs. Encoder-only LLMs utilize only the encoder component of the Transformer model [108]. These models are specifically designed to analyze and represent code or language context without generating output sequences, making them ideal for tasks that demand a detailed understanding of syntax and semantics. By employing attention mechanisms, encoder-only models encode input sequences into structured representations that capture essential syntactic and semantic information [56]. In the **software engineering** (**SE**) domain, encoder-only models such as CodeBERT [35], GraphCodeBERT [51], CuBERT [65], VulBERTa [53], CCBERT [158], SOBERT [59], and BERTOverflow [122] have been widely used.

Encoder-Decoder LLMs.Encoder-decoder models combine both the encoder and decoder components of the Transformer model, allowing them to handle tasks that require both understanding and generation of sequences. The encoder processes an input sequence, transforming it into a structured representation, which is then decoded to produce an output sequence. This structure makes encoder-decoder models versatile for tasks that involve translating, summarizing, or transforming

Table 1. Strengths and Weaknesses of Different LLM Categories for Vulnerability Detection

Category	Strengths	Weaknesses
Encoder-Only	Strong code understanding and static analysis (e.g., CodeBERT)	Poor at sequence generation and code modification
Encoder-Decoder	Balanced analysis and generation capabilities (e.g., CodeT5)	High compute cost; lacks task specialization
Decoder-Only	Excels at code generation and patching (e.g., GPT-3.5)	Limited in contextual understanding and dependency analysis

Table 2. Top 10 LLMs Used in Vulnerability Detection

Usages Ranking	LLM	Structure	Size
1	GPT-4	Decoder-only	Unknown
2	GPT-3.5	Decoder-only	Unknown
3	BERT	Encoder-only	109M
4	CodeBERT	Encoder-only	125M
5	CodeLlama	Decoder-only	7B, 13B, 34B, 70B
6	LLaMA	Decoder-only	7B, 13B, 70B
7	StarCoder	Decoder-only	15B
8	CodeT5	Encoder-Decoder	220M
9	Mistral	Decoder-only	7B
10	GraphCodeBERT	Encoder-only	125M

text or code. Prominent examples include PLBART [1], T5 [112], CodeT5 [133], UniXcoder [50], and NatGen [15].

Decoder-only LLMs. Decoder-only LLMs focus exclusively on the decoder component of the Transformer architecture to generate text or code based on input prompts. This approach leverages the model's capacity to interpret and extend context, enabling it to produce complex and coherent sequences by predicting subsequent tokens. Widely adopted for tasks that emphasize generation, such as vulnerability detection and code suggestion, decoder-only models excel in identifying relevant patterns and potential issues within code. Notable examples in this category include the GPT series (GPT-2 [111], GPT-3 [10], GPT-3.5 [105], GPT-4 [106]), as well as models tailored specifically for code in software engineering, such as CodeGPT [90], Codex [17], Polycoder [140], Incoder [40], CodeGen series[103], Copilot [47], Code Llama [113], and StarCoder [75]. [159]

In analyzing 58 studies on vulnerability detection, we identified 33 distinct LLMs used across various tasks. GPT-4 emerged as the most frequently used model, appearing in 29 instances, followed by GPT-3.5 with 25 mentions. Among the categories, encoder-only models represented 24.2% of total usage, with CodeBERT, GraphCodeBERT, UniXcoder, and BERT as prominent examples. Encoder-decoder models, including CodeT5, made up 8.7% of usage, serving dual roles in code generation and understanding. Decoder-only models, such as GPT series, CodeLlama, StarCoder, and WizardCoder, accounted for 67.1% of usage and were primarily applied in code generation tasks.

In addition, Table 2 presents the architecture and occurrences of the top 10 most commonly used LLMs in vulnerability detection research. It shows that most models for this task are decoder-only architectures, indicating a broader usage of this structure in detection tasks. Despite the general trend in the field, which often favors encoder-only architectures for understanding tasks, this table suggests that decoder-only models are also widely adopted for detection, likely due to their efficiency in processing and generating relevant sequences in code analysis.

Among all LLMs, the GPT series (especially GPT-4 series) consistently performs well due to its robust capabilities in understanding and generating code. GPT-4 is widely regarded for advanced

Table 3. Distribution of CVEs Across Different Software Systems

Software System	Publisher	CVE Number	Primary Language	Software Type
Gitlab	Gitlab	1068	Ruby	Application
Chrome	Google	3539	C++	Browser
Firefox	Mozilla	2700	C++	Browser
Android	Google	7215	Java	Operating System
MacOS X	Apple	3206	C	Operating System
Linux Kernel	Linux	5912	C	Operating System
Windows Server 2022	Microsoft	1607	C	Operating System

* Data collected until November 3, 2024 from NVD database

applications like vulnerability detection and code analysis, while GPT-3 and GPT-3.5 often serve as baselines or benchmarks in empirical studies. Specialized models such as CodeBERT and CodeT5 are frequently used for fine-tuned tasks involving code understanding and processing. Some research combines multiple models, such as pairing GPT-4 with GPT-3.5, to evaluate comparative performance or execute complementary tasks. This integrated approach, combining general-purpose LLMs with domain-specific models like CodeBERT, leverages the generalization power of LLMs and the task-specific precision of specialized models, resulting in improved performance and versatility.

Answer to RQ1

Recent trends show a shift from encoder-only models toward large decoder-only architectures like GPT and CodeLlama series in research. While encoder models still dominate non-fine-tuning studies (72.4%), decoder-only models account for 65% of fine-tuning experiments. Encoder-only and encoder-decoder architectures are increasingly positioned as baseline models for comparison.

3.3 RQ2. What Benchmarks, Dataset and Metrics Have Been Designed to Evaluate Vulnerability Detection?

In this section, we will start by examining the distribution of vulnerabilities across various programming languages and key software systems. We will then move on to discuss the benchmarks, datasets, and metrics commonly used in the field. Due to differences in design and feature, such as memory management in C/C++, unsafe deserialization in Python, and object injection and reflection in Java, different programming languages have different types of high-occurrence vulnerabilities. This is particularly significant, as many studies on vulnerability detection by LLMs focus on language-specific challenges [37, 77, 148]. Understanding these nuances is essential to evaluate and improve the effectiveness of vulnerability detection across different contexts.

We analyzed the CVE statistics across major software systems over the past five years (2019-2024), as shown in Table 3. 0

Table 3 reveals several interesting patterns in vulnerability distribution across different software systems. Operating systems (Android, MacOS X, Linux Kernel, and Windows Server) dominate the vulnerability landscape, followed by web browsers (Chrome and Firefox) and development platforms (Gitlab). According to CVE statistics [22], memory-related vulnerabilities have been the most prevalent type over the past five years. As memory-unsafe yet widely-used programming languages, C/C++ contributes to a significant number of memory corruption vulnerabilities, making vulnerability detection increasingly urgent. Based on our analysis of 56 selected papers, we collected statistics on all target programming languages, as shown in Figure 6.

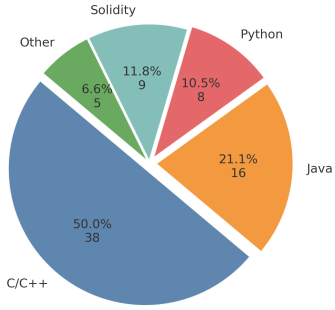

Fig. 6. Distribution of target programming languages in LLM-based vulnerability detection research.

Finding I

The research landscape shows a clear distribution in target programming languages: C/C++ dominates with 50% of studies, followed by Java at 21.1%. Solidity accounts for 11.8% of research due to its critical role in smart contracts and financial transactions. The remaining 16.6% covers other languages including Python, PHP, and Go.

C/C++ remains the primary focus in vulnerability detection, covering 50% of studies. This high proportion reflects memory-related vulnerabilities common in C/C++ projects. Java ranks second at 21.1%, partly due to its popularity in enterprise-level software and Android development (Table 3 shows Android contributes a large number of CVEs). Java's type system and bytecode format also provide detailed information for LLMs, and its web applications often face SQL injection, Cross-Site Scripting (XSS), or insecure deserialization. Solidity follows at 11.8%, as vulnerabilities in smart contracts directly threaten financial security on blockchain platforms. The remaining 16.6% includes languages like Python, PHP, and Go.

To fine-tune LLMs and measure their performance in vulnerability detection, researchers have introduced various datasets, including BigVul [33], CVEfixes [7], and Devign [161]. Each dataset targets different scales, from identifying whether a single function has a vulnerability to scanning an entire GitHub repository. This variation reflects the diverse needs of vulnerability detection tasks. However, existing datasets also suffer from several limitations, such as insufficient vulnerability diversity or low label quality. Consequently, many studies design their own datasets when constructing benchmarks to better adapt to the workflow of vulnerability detection. In Table 4, we further distinguish which works rely on publicly available datasets and which construct benchmarks based on self-designed datasets.

Function-level. Each data of these datasets contains the following attributes: function implementations (usually including both pre- and post-fix implementations, vulnerable flag (usually 1 for vulnerable and 0 for non-vulnerable). Frequently used datasets of this kind are BigVul [33] and Devign [161] (also referred to FFmpeg and QEMU dataset). These datasets are often used for fine-tuning large models and evaluating the ability of LLMs to detect vulnerability, but they are not much practical. The reason is that real-world vulnerabilities are usually caused by multiple functions across files.

File-level. Some datasets are structured not only in function level, but also in file level, like Juliet C/C++ [38] and Java [39] test suites. Some of the vulnerabilities in the Juliet test suites largely mimic the structure of real-world vulnerabilities, including, but not limited to, cross-file function

Table 4. Datasets/Benchmarks Frequently Used in LLMs for Vulnerability Detection

Dataset	Size*	Language	# Vuln. Types**	Scope	Source	labeled	Open-source
BigVul [33]	264,919 (11,823)	C/C++	91	function	real-world	✓	✓
CVEfixes [7]	12,107 (all)	C/C++, Python, PHP	180	commit	real-world	✓	✓
Juliet C/C++ [38]	64,099 (all)	C/C++	118	file	synthesized	✓	✓
D2A [156]	1,295,623 (18,653)	C/C++	N/A	function	real-world	✓	✓
DiverseVul [18]	349,437 (18,945)	C/C++	150	function	real-world	✓	✓
SARD [101]	> 5,000,000 (all)	C/C++, Java, PHP	150	file	mixed	✓	✓
Juliet Java [39]	28,281 (all)	Java	112	file	synthesized	✓	✓
Smartbugs-curated [37]	143 (all)	Solidity	10	contract	mixed	✓	✓
Smartbugs-wild [37]	47,518 (N/A)	Solidity	N/A	contract	real-world	✗	✓
DBGbench [12]	27 (all)	C/C++	6	application	real-world	✓	✓
Code Gadgets [80]	61,638 (17,725)	C/C++	2	function	mixed	✓	✓
Ponta2019 [109]	1,282 (all)	Java	6	commit	real-world	✓	✓
Vul4j [11]	79 (all)	Java	25	commit	real-world	✓	✓
Ghera [99]	69 (all)	Java	25	application	synthesized	✓	✓
Benchmark and Study with Self-Constructed Datasets							
Devign [161]	27,318 (12,460)	C/C++	N/A	function	real-world	✓	✓
ReVeal [16]	18,169 (1,664)	C/C++	N/A	function	mixed	✓	✓
SeVc [79]	420,627 (56,395)	C/C++	126	function	mixed	✓	✓
PrimeVul[29]	235,768 (6,968)	C/C++	140	function	real-world	✓	✓
MAGMA [57]	138 (all)	C/C++, Lua, PHP	11	repository	real-world	✓	✓
VulDeeLocator [78]	198,142 (40,450)	LLVM IR	4	function	mixed	✓	✓
Choi2017 [21]	14,000 (≈ 7,054)	C/C++	4	function	synthesized	✗	✓
Guo2024 [52]	13,532 (6,766)	C/C++	N/A	function	real-world	✓	✓
SVEN [58]	1,606 (all)	C/C++, Python	9	commit	real-world	✓	✓
Lin2017 [81]	6,486 (317)	C/C++	N/A	function	real-world	✓	✓
Ye2024 [147]	100 (all)	C/C++	1	application	real-world	✗	✗
ExtractFix [43]	7 (all)	C/C++	6	application	real-world	✓	✓
VulBench [44]	455 (all)	C/C++, decompiled code	9	function	mixed	✓	✓
VCMatch [132]	1,669 (all)	C/C++, Java and PHP	7	commit	real-world	✓	✓
Pan2023 [107]	6,541 (all)	C/C++, PHP, Java	78	commit	real-world	✓	✗
Ullah2023 [125]	228 (all)	C/C++, Python	8	function	mixed	✓	✓
Fang2024 [34]	15 (all)	Go, Java, PHP	9	application	real-world	✓	✗
CWE-Bench-Java [77]	120 (all)	Java	4	repository	real-world	✓	✗
Vulcorpus [70]	100 (all)	Java	10	function	synthesized	✓	✓
VjBench [138]	42 (all)	Java	23	commit	real-world	✓	✓
[148]	40 (all)	Python	10	function	synthesized	✓	✗
FELLMVP [91]	15,637 (820)	Solidity	8	contract	real-world	✓	✓
SolidiFI [45]	50 (all)	Solidity	7	contract	real-world	✓	✓
Ma2024 [93]	3,544 (1,734)	Solidity	5	function	mixed	✓	✗
SC-LOC [154]	1,369 (all)	Solidity	N/A	function	real-world	✓	✗
LLM4Vuln [121]	194 (97)	Java, Solidity	82	function	real-world	✓	✗
SmartFix [120]	361 (all)	Solidity	5	contract	mixed	✓	✗
Hu2023 [61]	13 (all)	Solidity	5	contract	real-world	✓	✓

*The number in parentheses represents the number of items that are vulnerable. For example, there are a total of 264,919 functions in the BigVul dataset, of which 11,823 are vulnerable.

**"N/A" indicates that the authors did not provide detailed information about the number of vulnerabilities in their papers.

calls or cross-file access to global variables. Such vulnerabilities with complex contexts provide a significant challenge for LLM to detect vulnerabilities. A test case of Juliet C/C++ test suite has been shown in Figure 7. We can see that in test case 501129, the files and lines where the vulnerabilities are introduced have been marked in the trace. This detail provides clues to LLMs' ability to detect vulnerabilities introduced by multiple files, but also suggests the need to improve LLMs' ability to detect across files.

Commit-level. Many open-source software use the GitHub platform and modify the source code by submitting a commit. Obviously, trusted maintainers can submit commits containing malicious changes that make the target software vulnerable [137]. So it is also necessary to apply vulnerability detection for each commit. CVEfixes [7] and Pan2023 [107] are both commit-level datasets. These datasets typically include the repository URL, commit hash (a unique identifier for each commit), and diff files (showing the differences before and after the commit). In this way, LLMs can analyze the code changes before and after the commit to determine whether the commit is vulnerable or not, and also fetch contextual information through the repository URL.

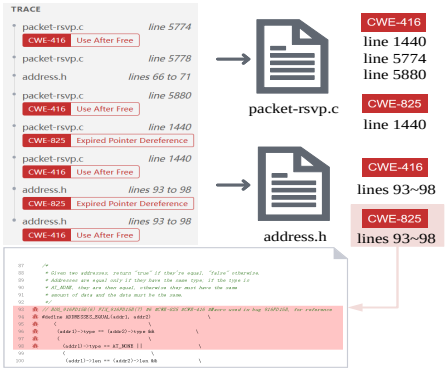

Fig. 7. The test case 501129 of Juliet C/C++ 1.3 test suite as a file-level dataset.

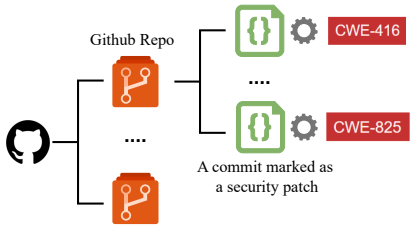

Fig. 8. An illustration for a commit-level dataset.

Repository- and application-level. These types of datasets are usually for vulnerability detection of the entire project. CWE-Bench-Java [77] is a repository-level datasets focusing on Java projects. Each repository comes with metadata about the vulnerability, such as CWE ID, CVE ID, remediation commits and vulnerability version. This makes analysis and validation more systematic and reliable. And Ghera [99] is an application-level datasets. Each item contains three applications: a vulnerable application that contains vulnerability X, a malicious application that can attack the vulnerable application using vulnerability X, and a secure application that has no vulnerability X. Each item comes with instructions to build and run the application to demonstrate the vulnerability and its exploitation, thus verifying the presence or absence of the vulnerability and exploitation. An example of commit-level dataset is shown in Figure 8.

There are also other kinds of datasets. For example, with the rise of blockchain, datasets for smart contract have been built. These datasets, like FELLMVP [91], contains many smart contracts (contract-level) with logical vulnerabilities (e.g., reentrancy attacks and integer overflow/underflow). There are also datasets that focus only on specific vulnerabilities. For example, Code Gadgets [80] only focuses on two types of vulnerabilities in C/C++ programs, buffer error vulnerability (CWE-119) and resource management error vulnerability (CWE-399). SolidFi [45] is only based on injection vulnerability. Frequently used datasets for vulnerability detection are shown in Table 4. The number in parentheses in the size column represents the number of items that are vulnerable.

Finding II

Current vulnerability datasets exhibit two major limitations: (1) Language imbalance - with C/C++ dominating at around 60% coverage while Java, despite being widely used in enterprise and Android development, lacks comprehensive datasets; (2) Scope gaps - there is a significant shortage of repository-level datasets that reflect real-world development scenarios where vulnerabilities often span multiple files and dependencies. This scarcity of realistic, large-scale repository datasets poses a critical limitation for practical applications of LLMs in vulnerability detection.

The evaluation of LLM-based vulnerability detection systems requires multiple metrics. These metrics can be categorized into three groups: classification metrics, generation metrics, and efficiency metrics.

3.3.1 Evaluation Metrics.

3.3.2 Classification Metrics. Vulnerability detection systems commonly use several standard metrics: Accuracy ($\frac{TP+TN}{TP+TN+FP+FN}$) measures overall correctness; Precision ($\frac{TP}{TP+FP}$) indicates true positive rate; Recall ($\frac{TP}{TP+FN}$) shows vulnerability coverage; and F1-Score ($2\times\frac{Precision\times Recall}{Precision+Recall}$) balances precision and recall. For imbalanced datasets, Matthews Correlation Coefficient (MCC) is also useful:

$$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}. \tag{1}$$

3.3.3 Generation and Explainability Metrics. To evaluate the quality of generated vulnerability descriptions, Alam et al. and Ghosh et al. [2] and [46] employ BLEU and ROUGE metrics. BLEU considers brevity penalty and n-gram precision, while ROUGE measures overlap between generated and reference texts. These metrics help assess both the accuracy and explainability of LLM-based vulnerability detection systems.

Answer to RQ2

Most benchmarks and datasets for LLM-based vulnerability detection focus on function-level or file-level scope, with C/C++ as the dominant target language. Classification metrics, like accuracy and precision, are widely used, with Matthews Correlation Coefficient (MCC) adopted for imbalanced datasets. Metrics such as BLEU and ROUGE assess the quality of generated descriptions, while execution time evaluates efficiency. However, current datasets are limited by their focus on C/C++ and lack of repository-level data. These gaps hinder LLMs' ability to generalize across languages and detect complex, multi-file vulnerabilities. Future research should create diverse, large-scale datasets to simulate the real-world scenarios.

3.4 RQ3. What Techniques are Used in LLMs for Vulnerability Detection?

Currently, LLM-based vulnerability detection faces several key challenges: (1) data leakage leading to inflated performance metrics, (2) difficulty in understanding complex code context, (3) positional bias in large context windows causing information loss, and (4) high false positive rates and poor performance on zero-day vulnerabilities. Researchers have conducted extensive studies to address these challenges. This section summarizes and discusses current techniques applied to LLM-based vulnerability detection.

3.4.1 Code Data Preprocessing. Code processing techniques serve two primary objectives: (1) optimizing the utilization of LLMs' limited context window to improve efficiency, and (2) enhancing LLMs' comprehension of semantic information within the code to improve vulnerability detection capability.

Abstract Syntax Tree Analysis. **Abstract Syntax Tree** (**AST**) provides a hierarchical representation of program structure, where code elements are organized into a tree format based on their syntactic relationships [135]. This structural representation eliminates non-essential syntax details while preserving the semantic relationships between code components. Figure 9 represents the AST for a code snippet. AST applications in vulnerability detection can be categorized into several primary approaches: code segmentation and structural representation, where ASTs parse code into function-level segments for efficient processing within LLMs' context limits, as demonstrated by Zhou et al. [157] and Mao et al. [95]; semantic enhancement, where ASTs are integrated with natural language annotations to form **structured comment trees** (**SCT**), as implemented in SCALE framework [136] to capture vulnerability patterns beyond syntactic relationships; multi-graph analysis,

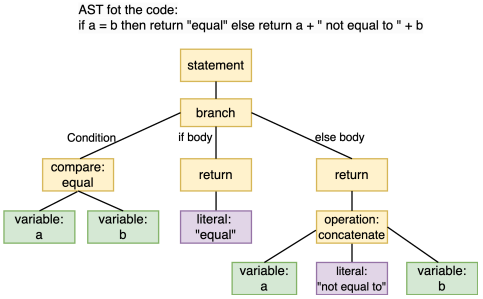

Fig. 9. An Example Illustrating AST

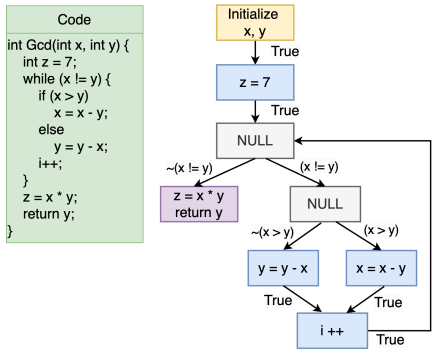

Fig. 10. An example illustrating DFG.

where ASTs are combined with **control flow graphs** (**CFG**) and **data flow graphs** (**DFG**) to provide comprehensive code structure analysis, as shown in DefectHunter [130]; pattern detection integrated with **graph attention networks** (**GATs**), exemplified by VulnArmor [119], GRACE [89], and [94]; and error localization for code evolution as demonstrated in [154]. Empirical evaluations across diverse datasets including FFmpeg, QEMU, and Big-Vul demonstrate that AST-augmented approaches significantly improve vulnerability detection performance.

Data/Control Flow Analysis. AST lacks a representation of the data and control flow of a program. Thus, in some papers [69, 77, 84, 87, 89, 121], DFG and CFG have been applied to help LLMs understand the interprocedural data flow and control flow in a program. Figure 10 is a concise example illustrating a Java method and its corresponding DFG. DFG summarizes the possible execution paths in a program, using nodes (or basic blocks, i.e., statements that are executed sequentially without any branching operation) to represent program constructs, and edges to represent the flow of data. CFG has the same basic blocks as nodes, but edges are used to represent the branching operations between basic code blocks. There are two main usages of DFG/CFG like providing extra contextual information in prompt and knowledge base. Combine source code with its DFG and CFG into prompt will result in a significant improvement in LLm's performance in identifying vulnerabilities [69, 84, 87, 89]; Sun et al. has proposed that DFG and CFG can be used in knowledge base, with graph-based similarity-search algorithm, to provide LLMs with the information of code segments with similar data and control flow structure [121]. Except for DFG and CFG, call graph has been used in vulnerability detection [91] to give LLM more information about dependencies between functions.

Retrieval-augmented Generation. **Retrieval Augmented Generation** (**RAG**) enhances the capabilities of LLMs by integrating an information retrieval system that provides extra related information to LLMs [73]. LLMs receives user input and applies a searcher to find relevant documents or pieces of information from a knowledge base. The retrieved information is combined with the original prompt to generate a response. In this way, RAG can solve the problem of insufficient knowledge of LLMs in certain domains, illusions and that LLMs cannot update data in real time. Many papers have discussed how to choose the right knowledge as a high priority when building RAG for LLMs. [13, 68, 70, 98, 121]. Cao et al. directly use CWE database as external knowledge [13]. Many papers focused on combining code snippets, static analysis results with documentation of corresponding vulnerabilities as knowledge [68, 70, 98]. In addition to the knowledge mentioned above, Sun et al. used GPT-4 to summarize existing knowledge and thus create two knowledge bases

(VectorDB with Vulnerability Report and Summarized Knowledge) [121]. RAG has been proven to improve LLM's ability to detect vulnerabilities [68].

Program Slicing. Program slicing technique has been used to reduce vulnerability-irrelevant lines of code and keep critical lines related to trigger vulnerability [14, 94, 110, 154]. Purba et al. apply program slicing technique to extract code snippets for buffer-overflow detection [110]. These code snippets usually contains key functions, like `strcmp` and `memset`, and statements related to call these functions. Cao et al. use program slicing in similar way [14]. They first find all potential vulnerability triggers, and apply program slicing technique to collect all statements related to these triggers. While Purba et al. [110] and Cao et al. [14] only use the program slicing technique for code preprocess, Zhang et al. proposes to fine-tune LLMs with sliced code to improve the performance of LLMs vulnerability detection [154]. Instead of setting explicit slicing criteria, LLMs learns to segregate vulnerable lines of code from a given function during training. This approach helps the model to focus on relevant parts of the code and identify vulnerabilities more accurately. The program slicing technique has been shown to improve LLMs' ability to detect vulnerability [110, 154].

LLVM Intermediate Representation. By converting source code to LLVM **Intermediate Representation** (**IR**) [72], analysis and detection methods do not need to be specifically adapted to different programming languages. This improves the versatility of vulnerability detection methods, and LLVM IR also preserves program structure and semantics, making it easier for LLM to analyze potential dependencies in the code. But the downside is obvious: LLVM IR does not work with Java and Javascript. To make the approach generalizable to programming languages, the authors converted the source code to LLVM IR and trained LLMs on these IR [94].

We can see that in the field of LLMs in vulnerability detection, the techniques mainly comes from traditional software analysis and LLM research. And basically the outputs of these techniques are used as part of the prompts to evaluate the LLMs' vulnerability detection capabilities. These approaches not only optimize the efficiency of LLMs' context window utilization, but also improve its understanding of potential vulnerabilities by preserving or extracting semantic information from the code. However, it also inevitably increases token consumption, and there is also the possibility that too much prompt content may reduce the ability of LLMs to detect vulnerabilities.

Finding III

Our analysis reveals that 41.3% of studies employed code processing techniques - including graph representations, Retrieval-Augmented Generation (RAG), and code slicing - to better utilize LLMs' limited context windows. While these approaches show modest improvements over direct code prompting, their effectiveness diminishes significantly when dealing with complex, cross-file vulnerabilities. Notably, as larger LLMs (like the GPT series) emerge, the performance gains from the models themselves tend to outweigh those from code processing techniques. This suggests that while current code semantic processing methods offer benefits, developing more effective ways to represent complex code context remains a crucial challenge.

3.4.2 Prompt Engineering Techniques. This is one of the most widely used strategies for optimizing LLM-based vulnerability detection systems, as it enables precise control over model responses by tailoring input prompts. Our supplemental file provides a brief summary of the most commonly adopted prompt engineering techniques for vulnerability detection, together with representative papers. It is worth noting that here "representative papers" refers to works that provide clear and illustrative prompt examples (often accompanied by diagrams) rather than those that merely report strong numerical results. We argue that well-documented case studies of prompt design are more

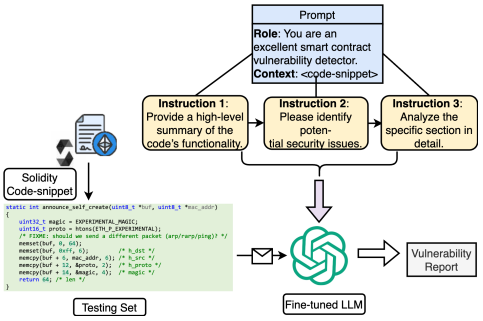

Fig. 11. An Instance of Multi-level Prompting for Vulnerability Detection

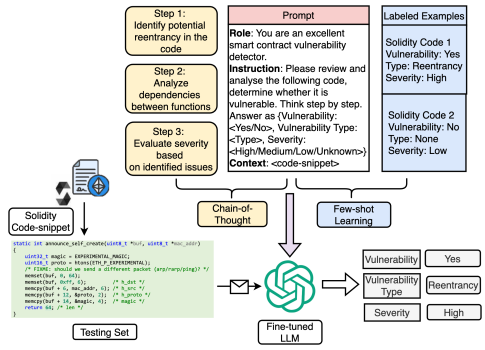

Fig. 12. Principles of chain-of-thought and few-shot Learning

valuable for advancing this line of research, as they offer actionable insights for practitioners and future research.

Chain-of-Thought Prompting. **Chain-of-Thought** (**CoT**) Prompting[134] is a technique where LLMs are guided to follow step-by-step instructions to enhance reasoning accuracy before generating a final output. Figure 12 illustrates the CoT reasoning process for vulnerability detection. In LLM-based vulnerability detection, CoT prompting involves instructing the model to first summarize the functionality of the given code, then assess potential errors that might introduce vulnerabilities, and finally determine if the code is vulnerable. This structured prompt strategy has been shown to improve precision and recall in vulnerability detection tasks by helping the model reason through complex code in a more organized manner. However, while CoT prompting often enhances precision, its impact on recall can vary across different scenarios [28, 82, 93, 121].

Recently, especially since 2025, we have observed significant improvements in the overall capability of LLMs, particularly in code understanding and reasoning. Consequently, an emerging line of research has begun leveraging LLMs to score potentially vulnerable code blocks, thereby prioritizing the most suspicious regions of code or selecting the most effective fuzzing seeds for more efficient vulnerability detection. For instance, [93] and [104] employ LLMs to generate "confidence scores," where a lower score (e.g., 1) indicates relatively safe code and a higher score (e.g., 10) suggests highly suspicious or dangerous code. Such scores are then used to guide vulnerability detection or to steer fuzzing towards generating more reasonable and effective new inputs.

Few-shot Learning. In LLM-based vulnerability detection, **few-shot learning** (**FSL**) enables models to leverage a small set of labeled examples within prompts to improve task-specific performance. In this approach, vulnerability detection can be enhanced by embedding classification standards, such as CWE, directly into the prompt [44]. By incorporating CWE vulnerability categories-complete with numbers and descriptive names-the model gains contextual knowledge that aids in identifying and classifying vulnerabilities accurately. Figure 12 also illustrates the principle of few-shot learning, where the model is provided with labeled examples (e.g., Solidity Code 1 and Solidity Code 2) to understand the task before analyzing a new testing set. These examples, combined with the task-specific prompt, guide the fine-tuned LLM to generate accurate outputs.

Hierarchical Context Representation. Hierarchical context representation is a technique used to manage the context length limitations of LLMs when analyzing extensive codebases. In vulnerability detection, code can be organized hierarchically into modules, classes, functions, and statements. By representing the code in this hierarchical manner, the LLM can process and analyze

the code at different levels of abstraction. This approach allows the model to focus on higher-level structures before delving into detailed code segments, effectively managing the context and improving the detection of vulnerabilities within the constraints of LLMs's maximum input length.

Multi-level Prompting. The multi-level prompting strategy involves breaking down the vulnerability detection task into multiple prompts, each targeting a specific level of analysis. Instead of presenting the entire code and task in a single prompt, the strategy divides the process into stages. For example, the first prompt may ask the LLM to provide a high-level summary of the code's functionality. The second prompt might focus on identifying potential security issues, and subsequent prompts could request detailed analyses of specific sections. This layered approach helps the LLM to systematically process complex code, enhancing its ability to detect vulnerabilities by focusing on one aspect at a time. Figure 11 illustrates an instance of multi-level prompting.

Multiple Prompt Agents and Templates. This technique employs several specialized prompt agents, each designed with a specific template to perform distinct roles in the vulnerability detection process. For instance, one agent might be tasked with code summarization using a template that guides the LLM to extract key functionalities. Another agent could focus on vulnerability identification, utilizing a template that prompts the model to look for common security weaknesses. By using multiple agents with tailored templates, the system leverages the strengths of each specialized prompt, leading to more accurate and comprehensive vulnerability detection results.

In general, prompt engineering effectiveness varies with model size. Small models benefit from few-shot learning and structured prompting to compensate for limited capabilities, while large models perform better with chain-of-thought prompting and zero-shot approaches due to their stronger generalization abilities and domain knowledge.

Finding IV

As LLMs' inherent capabilities grow, their context window capacity expands accordingly. Chain-of-Thought (CoT) prompting emerges as the dominant approach for large models (>10B parameters), with 100% of recent studies adopting CoT to enhance generation. For smaller models with limited text processing capacity, zero-shot or minimal few-shot approaches prove more effective, as CoT and extra few-shot examples may cause irrelevant output.

3.4.3 Fine-Tuning. Fine-tuning helps LLMs learn specific tasks better. It works by training pre-trained models again with new data for these tasks. A comparison among studies in fine-tuning is shown in Tables 5 and 6. Fine-tuning is important for three main reasons [146, 159]: (1) security problems in code follow special patterns that LLMs must learn to find, (2) computer code is different from normal text, so LLMs need to learn how to read and understand code better, and (3) finding security problems needs to be very accurate—missing real problems or reporting false ones can both cause serious issues. As shown in Table 5, approximately 30% of studies choose to fine-tune existing LLMs as their primary proposed method.

Full Fine-tuning (FFT). FFT updates all model parameters during training. Due to computational constraints, most research utilized models with fewer than 15 billion parameters, such as CodeT5, CodeBERT, and UnixCoder. Ding et al.[29] experimented with five models under 7B parameters, achieving only 0.21 F1-score even when training and validating on PrimeVul. Guo et al.[52] utilized CodeBERT with FFT for 50 epochs, achieving 0.099 F1-score on PrimeVul but 0.66 F1-score on the Choi2017 dataset. Haurogne et al. [54] achieved 0.69 F1-score on the DiverseVul dataset, while Purba et al.[110] achieved 0.73 F1-score in buffer overflow detection.

Table 5. Fine-Tuning Methods in LLM-Based Vulnerability Detection

Paper	Target Language	FT Method	Main Dataset	Model (F1-Score)	Open-source
Alam et al. [2]	Solidity	PEFT	VulSmart[2]	GPT-4o-mini (0.99)	✗
Boi et al. [8]	Solidity	PEFT	VulHunt[9]	Llama2-7B-chat-hf	✗
Cao et al. [14]	PHP	PEFT	RealVul[14]	GPT-4 (0.58)	✓
Ding et al. [29]	C/C++	FFT	PrimeVul[29]	UnixCoder (0.21)	✓
Du et al. [30]	C/C++	FFT+IFT	Devign[161]	CodeLlama (0.71)	✓
Ghosh et al. [46]	C/C++	DAPT	NVD	MPT-7B (0.93)	✗
Gonçalves et al. [48]	C/C++	FFT&PEFT	CVEFixes[7]	NatGen (0.53)	✓
Guo et al. [52]	C/C++	FFT&PEFT	Devign[161]	CodeLlama-7bB (0.97)	✗
Haurogné et al. [54]	C/C++	FFT	DiverseVul[18]	BERT (0.69)	✗
Liu et al. [85]	C/C++	FFT	SARD [101]	Llama-3-8B	✓
Luo et al. [91]	C/C++	PEFT	Liu2023[86]	Gemma-7B (0.90)	✗
Ma et al. [93]	Solidity	PEFT	Ma2024[93]	CodeLlama-13B (0.91)	✗
Mao et al. [95]	C/C++	PEFT+IFT	SeVC[79]	CodeLlama-13B (0.92)	✗
Purba et al. [110]	C/C++	FFT	Code Gadget[80]	Davinci (0.73)	✗
Sakaoglu et al. [115]	HTML/JavaScript	FFT	Sakaolu[115]	DistilRoBERTa (0.82)	✗
Shestov et al. [118]	Java	PEFT	CVEFixes[7]	WizardCoder (0.71)	✗
Taghavi et al. [160]	C/C++/Java	PEFT	Mixed Dataset	GPT-4 (0.90)	✗
Wang et al. [130]	C/C++	Model Innovation	QEMU	UnixCoder (0.71)	✓
Wen et al. [136]	C/C++	FFT	QEMU	UnixCoder (0.65)	✗
Yang et al. [142]	C/Java/Python	PEFT	LLMAO[142]	CodeGen-16B	✓
Yin et al. [149]	C/C++	FFT	Big-Vul[33]	DeepSeek-Coder-6.7B (0.270)	✗
Zhang, C. et al. [152]	C/C++	PEFT	Zhang2024[152]	Llama-7B (0.85)	✗
Zhang, J. et al. [154]	C/C++/Solidity	FFT	Big-Vul[33]	CodeLlama-7B (0.82)	✗
Zhou et al. [157]	C/Java/Python	FFT&PEFT	Zhou2024[157]	Llama-3-8B	✗

Table 6. Comparisons of Different Fine-Tuning Methods in LLM-Based Vulnerability Detection

Dimension	Method	Properties
Parameter Scale	Full Fine-tuning	Updates all parameters for high adaptation capability
Parameter Scale	PEFT	Updates subset of parameters for resource efficiency
Learning Approach	Discriminative	Binary/multi-class classification for precise detection
Learning Approach	Generative	Sequence-to-sequence learning with rich output format

Parameter Efficient Fine-tuning (PEFT). PEFT methods modify only a subset of parameters while keeping most pre-trained weights frozen. Adapters introduce additional trainable layers between original model layers, with Yang et al. [142] achieving 60% Top-5 accuracy in fault localization. LoRA represents weight updates as low-rank decompositions, with studies like Du et al. [30] achieving 0.72 F1-score and Guo et al. [52] reaching 0.97 F1-score on their respective datasets. QLoRA combines parameter quantization with LoRA, as demonstrated by Boi et al.[8] achieving 59% accuracy with lower memory usage.

Discriminative Fine-tuning. For a token sequence $X = \{x_1, x_2, \ldots, x_L\}$, the model processes it to output vulnerability labels. Zhang et al. [154] and Yin et al. [149] demonstrated that a fine-tuned CodeLlama achieves a 0.62 F1-score improvement compared with its non-fine-tuned counterpart.

Generative Fine-tuning. This approach enables sequence-to-sequence learning for generating structured outputs like vulnerability descriptions or vulnerable line identification. Yin et al.[149] showed that fine-tuning pre-trained LMs outperforms fine-tuning LLMs in generative tasks, with CodeT5+ achieving a ROUGE score of 0.722 compared with DeepSeek-Coder 6.7B's 0.425.

Finding V

Fine-tuning enhances LLM-based vulnerability detection through full and parameter-efficient methods (PEFT). Large models (>10B) with PEFT achieve optimal results, while base models like GPT-4 and CodeLlama deliver F1 scores near 0.9. Discriminative strategies excel in precise detection, requiring datasets with at least 10K samples. However, computational limits and dataset quality remain critical challenges.

Answer to RQ3

LLM-based vulnerability detection techniques fall into three categories. First, code preprocessing-such as AST analysis, data/control flow analysis, RAG, and program slicing-enhances context utilization but struggles with complex, cross-file vulnerabilities. Second, prompt engineering-like CoT prompting, few-shot learning, and specialized agents-improves accuracy, with large models (>10B) benefiting from chain-of-thought methods, while smaller models favor simpler prompts. Finally, fine-tuning-both full and parameter-efficient approaches like LoRA-achieves near 0.9 F1-scores, particularly in larger models. As models advance, their inherent strengths may surpass preprocessing benefits, highlighting the need to address complex contexts and cross-file vulnerabilities.

3.5 RQ4. What are the Challenges that LLMs are Facing in Detecting Vulnerabilities and Potential Directions to Solve Them?

The field currently faces four major challenges, along with corresponding potential directions, as illustrated in Figure 13. First, researchers struggle to obtain high-quality datasets. Second, LLMs show reduced effectiveness when dealing with complex vulnerabilities. Third, these models have limited success in real-world repository applications. Fourth, the models lack robust generation capabilities. Multiple studies confirm these challenges as the main barriers to progress. The following sections examine each challenge in detail.

Challenge 1: Limited scope of research problems: Current research focuses primarily on determining whether a given code snippet is vulnerable or not. In this survey, approximately 40 studies (83%) concentrate on the analysis of isolated code snippets, where LLM performance often exceeds the results observed in real-world code detection scenarios [44]. While this provides a controlled environment for evaluation, it overlooks the complexities present in practical applications, such as analyzing entire codebases or addressing vulnerabilities that emerge during collaborative development. This indicates that research focused solely on isolated functions or snippets has limited usefulness for real-world scenarios.

Potential Directions: Beyond analyzing isolated code segments, future research should be structured around the following key problems in real-world development:

Research problems categorized by code evolution in development:

— **Full-scale/Incremental Detection:** Full-scale detection requires analyzing entire codebases across multiple files, while incremental detection focuses on new code commits. Current LLMs excel at analyzing isolated code segments but struggle with broader contexts [63]. As a result, LLMs typically assist static analysis tools or support fuzzing tasks [92, 143, 144] rather than performing standalone analysis. For commit-level detection, existing methods combine commits with static analysis results [74, 145], but may fail when encountering APIs outside their training corpus [63].

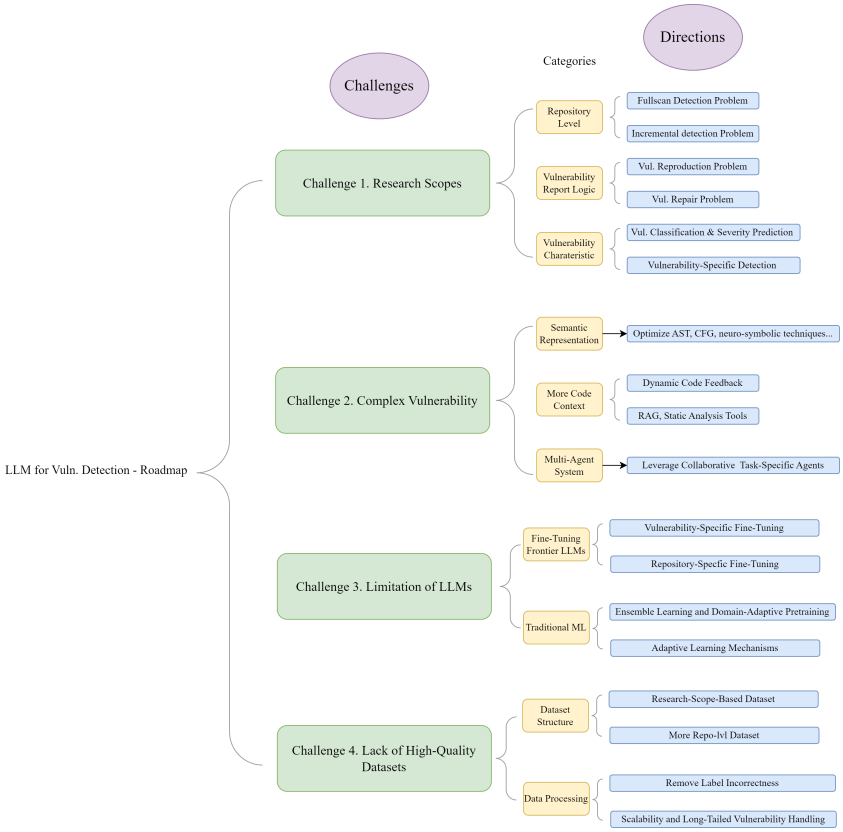

Fig. 13. Challenges and potential directions.

Research problems categorized by vulnerability report workflow:

— **Reproducing Vulnerabilities**: Vulnerability reproduction will be essential in future research and a key to reducing false positives. For each detected vulnerability, LLMs should attempt reproduction using input drivers such as fuzzers [63]. By generating inputs that trigger potential vulnerabilities, LLMs can provide evidence of vulnerability existence [139, 143]. This approach validates the detection and ensures findings are actionable, thus improving vulnerability report reliability.

— **Vulnerability Repair**: While many studies discussed vulnerability repair, practical implementation in real-world projects remains challenging [159]. Successful vulnerability repair in production environments must meet several criteria:
  – Repaired code must pass all existing tests
  – Repaired code must prevent vulnerability reproduction
  – Repaired code should not introduce new vulnerabilities

  Current limitations in dataset quality and LLM capabilities hinder effective vulnerability repair validation. While LLMs can identify vulnerable code, they often misidentify vulnerability locations, leading to incorrect explanations and repairs. The ability to generate proof-of-concept exploits and simulate program operations would improve validation, but this requires significant advances in LLM capabilities [71].

Research problems categorized by vulnerability characteristics:

— **Specialized Vulnerability Detection:** The development of targeted detection approaches for specific vulnerability categories represents a significant challenge. Current research [43] demonstrates that LLMs exhibit varying capabilities across different vulnerability types - achieving high accuracy in detecting Out-of-bounds Write vulnerabilities (CWE-787) while performing poorly with Missing Authorization issues (CWE-862). This performance variation necessitates specialized detection mechanisms for distinct CWE categories, particularly for high-frequency vulnerabilities such as memory-related issues. The lack of category-specific research and datasets has left this critical area largely unexplored.
— **Vulnerability Classification and Severity Assessment:** Alam et al. [2] and Gao et al. [43] highlight two fundamental challenges: First, accurately categorizing detected vulnerabilities according to established frameworks (e.g., CWE) remains difficult. This classification problem is essential to practical development workflows, as different vulnerability types require distinct remediation approaches. Second, predicting vulnerability severity levels directly influences remediation priorities and timelines, with high-risk vulnerabilities requiring expedited mitigation.

Finding VI

When selecting experimental research problems, researchers should prioritize addressing real-world challenges in software vulnerability detection. For instance, research can be categorized by codebase analysis methods, such as full-scale scanning or incremental scanning, focusing on comprehensive or commit-level vulnerability detection. Additionally, based on the logical workflow of vulnerability reporting in real-world development, research can be divided into vulnerability discovery, reproduction, and repair. Beyond these, other feasible directions include vulnerability classification, severity prediction, and targeted detection for specific vulnerability types.

Challenge 2: Complexity of Representing Vulnerability Semantics. Vulnerability patterns are often very complex [76]. More than 95% of studies report that code complexity-such as external dependencies, multiple function calls, global variables, and complicated software states-makes it hard to detect vulnerabilities. We can measure this complexity by lines of code or cyclical complexity [123], and visualize it using program dependency graphs (DFGs) or call graphs. However, most methods focus on single code blocks at the function or file level [30, 154], which is not very helpful for large projects. When dealing with complex projects, LLMs often have limited input and face much "unseen code" [63].

In simpler situations-like a single function of about 500 lines from synthetic datasets-LLMs can detect vulnerabilities well, even in zero-shot settings [14, 139]. But many studies show that more information is needed to handle larger projects [3, 44], especially when the online corpus is sparse. In such cases, we must provide extra documents and specifications [153]. Also, some functions rely on their callers for protection, so analyzing them alone may lead to incorrect conclusions. We need to give LLMs enough context to identify vulnerabilities accurately.

Potential Directions: Two core approaches can address this challenge. The first approach focuses on enabling LLMs to read more code. This increases their understanding of the entire repository. The second approach emphasizes using abstract representations to simplify code semantics. This enhances LLMs' comprehension of code structure and behavior.

— **Dynamic Code Knowledge Expansion:** Through feedback loops and adaptive mechanisms, LLMs should be enabled to freely access and understand repository code [144, 153]. This would address high false positive rates by providing broader context for vulnerability analysis.

- **Optimized Code Representation:** Studies [69, 84, 87, 89, 121] utilize AST, CFG, and DFG representations to reduce token counts for limited context windows. While current implementations have not significantly improved detection accuracy, future research opportunities include sophisticated semantic processing, multi-method integration, better context preservation, and hybrid graph representations.
- **Specialized LLM Agents** Research explores optimization through specialized LLM agents. Studies [131] demonstrate that task division among multiple agents increases output robustness. Each agent specializes in specific aspects of vulnerability detection. Prompt engineering research [87] evaluates zero-shot, few-shot, and chain-of-thought approaches for detection accuracy. However, code complexity introduces challenges. Multiple agents show accuracy degradation with complex code. Few-shot and chain-of-thought methods cannot provide sufficient additional context.
- **Integration with External Tools** External tools provide important support mechanisms. LangChain improves efficiency through simplified and asynchronous LLM calls. **Retrieval-Augmented Generation (RAG)** gains popularity due to its cost-effectiveness and efficiency. Studies [13, 31] implement RAG to vectorize code contexts and enhance detection through LLM-based retrieval. However, code contexts differ fundamentally from natural language. This difference necessitates specialized approaches for code semantic extraction, storage, and comparison.

  These optimization techniques could potentially bridge the gap between LLMs' current capabilities and the complex requirements of real-world vulnerability detection in large codebases.

Finding VII

The complexity of vulnerabilities indicates that vulnerable code often involves intricate control flows. Addressing this requires improving LLMs' ability to efficiently store and process code information. Researchers can use retrieval-augmented generation (RAG) or similar tools to dynamically expand knowledge. Code semantics can be compressed using control flow graphs (CFGs), abstract syntax trees (ASTs), or neural-symbolic methods. Additionally, specialized agents optimized for specific tasks can be employed within vulnerability detection systems. These approaches enhance the efficiency and effectiveness of LLMs in handling complex code structures.

Challenge 3: Intrinsic Limitations of LLMs. Detection solutions must maintain robustness against data changes and adversarial attacks [30]. However, research by Yin et al. [149] reveals that LLMs lack this robustness. They show vulnerability to data perturbations. These findings indicate the need for more reliable approaches.

Additionally, LLMs need better explainability and consistency for real-world applications. Haurogne et al. and Du et al. [30, 54] demonstrate that LLMs produce inconsistent vulnerability explanations. They cannot guarantee correct explanations in every instance. The outputs show randomness across different runs. Even when LLMs correctly identify vulnerable code, they often fail to provide accurate vulnerability explanations. This limitation creates significant problems for subsequent repair and review processes. Current research in this area remains insufficient.

Future research should focus on these key areas: improving accuracy, enhancing robustness, and increasing output reliability and explainability. These improvements will make LLM-based solutions more practical for real-world use.

Potential Directions: The core to this challenge is to improve the inherent vulnerability detection ability of LLMs. Researchers may focus on training new models or fine-tuning LLMs.

- **Fine-tune Frontier LLMs:** Recent findings on scaling laws [66] indicate that larger decoder-only language models, such as GPT-4 and Claude3.5-Sonnet, can achieve systematically improved performance as model size, training data, and compute are scaled up. Improvements in model capabilities enhance vulnerability detection, classification, and explanation. Research by Alam et al. [2] shows that GPT-4 and GPT-3.5 achieve higher detection accuracy than earlier models like Llama2 and CodeT5 under identical prompts. The release of GPT-O1, with its visible reasoning process, suggests improvements in both detection capability and output explainability.
  Fine-tuning approaches show promise. Several studies [2, 14, 30, 52, 93, 96] fine-tune open-source models like CodeLlama and CodeBERT. These achieve results comparable to general LLMs. However, dataset limitations present challenges. Research [129] indicates that practical applications require datasets of at least 100,000 examples. This creates significant training cost barriers. Researchers can enhance vulnerability detection performance through the following approaches:
  – Vulnerability-specific Fine-tuning: Fine-tune models on specific vulnerability types. Research [2, 14, 30] shows models trained on specific vulnerability categories (e.g., memory-related issues, injection flaws) achieve higher detection accuracy. This targeted approach allows models to learn deeper patterns within each vulnerability type.
  – Repository-adaptive Fine-tuning: Adapt models to specific codebases through fine-tuning on repository-specific data. Studies [53, 94] demonstrate this approach improves detection accuracy by helping models understand project-specific coding patterns and architecture. This method benefits large, complex projects with unique coding conventions.
- **Ensemble Learning and Adaptive Learning Mechanisms:** Combining predictions from multiple models effectively reduces false positives and improves detection accuracy. To address the dynamic nature of security threats and enhance model robustness, adaptive learning [97] mechanisms allow continuous knowledge updates through feedback loops and periodic retraining. Advanced optimization techniques can further improve prompt configurations and learning rates, ensuring reliability in real-world applications.

Finding VIII

Enhancing robustness and explainability in LLMs is essential for effective vulnerability detection. Fine-tuning on specific vulnerability types, such as memory issues or injection flaws, improves detection accuracy by focusing on targeted patterns. Repository-adaptive fine-tuning helps models learn project-specific coding conventions, further increasing accuracy. Adaptive learning mechanisms, incorporating feedback loops and periodic updates, ensure LLMs remain robust against evolving threats. These methods address LLM limitations and improve their real-world applicability.

Challenge 4: Lack of High-Quality Datasets. High-quality vulnerability benchmark datasets remain scarce. Current datasets face several problems. These include data leakage, incorrect labels, small size, and limited scope [18, 98, 138].

Dataset Incorrectness: A critical issue in these challenges is the incorrect labeling of vulnerabilities, which harms the reliability and effectiveness of datasets. Automated collection methods [102] can gather large amounts of data quickly but cannot ensure correct labels without human review, leading to many mislabeled or inaccurately annotated samples. Research [138] also shows frequent data leakage, as LLMs often train on sources like GitHub, old software versions, and external libraries with inadequate version control or deduplication. As a result, models may encounter the same test

data seen during training, inflating performance metrics and undermining real-world validity. In conclusion, a high-quality dataset for vulnerability detection should meet several requirements.

- **Accurate Labels.** Since labels are crucial in supervised learning, incorrect annotations can lead to serious issues in production environments.
- **Minimal Data Leakage.** Large-scale LLMs trained on broad codebases risk seeing identical vulnerabilities during testing. Countermeasures include code obfuscation, synthesis, and using updated datasets.
- **Comprehensive Annotations.** For repository-level data, providing call sequences and control flows that reproduce the vulnerability, as well as detailed descriptions, helps LLMs create more reliable detection reports.

Potential Directions: The key to addressing this challenge lies in researchers' focus on constructing datasets based on specific research scopes. As discussed in Challenge 1, different research problems in LLM-based vulnerability detection require distinct types of datasets, all of which currently lack sufficient accurate data or cases. Researchers should approach dataset construction with targeted focus on specific research problems. Researches can be developed on:

- **Dataset Quality and Scope Enhancement.** Research can focus on developing smaller, high-quality test sets to effectively measure progress in vulnerability detection. One approach combines existing verified samples from multiple studies [2, 85, 163]. This creates a reliable test benchmark that the research community can maintain and expand over time. Moreover, recent advances in LLMs, particularly GPT-4o with its 128k context window, enable comprehensive repository-level vulnerability analysis, allowing researchers to detect and repair vulnerabilities across entire codebases rather than just at function-level.
- **Scalability and Long-Tailed Vulnerability Handling:** Handling long-tailed distributions of vulnerability types requires both scalable models and data augmentation techniques [27, 83]. Generating synthetic samples for rare vulnerability types can improve LLMs' ability to detect low-frequency events. Integrating structured information, such as CWE classifications, can further enhance the model's capability to prioritize and address critical vulnerabilities effectively [5].

Finding IX

High-quality datasets are essential for advancing LLM-based vulnerability detection. Repository-level datasets with detailed annotations, including call sequences and control flows, enhance real-world applicability. Targeted datasets aligned with specific research scopes address distinct detection challenges. Synthetic data generation mitigates data leakage and handles rare vulnerability types. Combining verified samples with scalable data augmentation ensures robust benchmarks for repository-wide vulnerability detection.

Answer to RQ4

The main challenges in LLM-based vulnerability detection include research scopes, dataset quality, vulnerability complexity, and model robustness. Key research directions involve improving model capabilities, developing advanced usage methods, enhancing datasets, strengthening detection robustness, and specializing vulnerability detection approaches. There is still a long way to go.

4 Limitations

Several factors may affect this survey's comprehensiveness. First, approximately 60% of research in LLM-based vulnerability detection appears as preprints on arXiv. This reflects the field's emerging nature. Second, terminology variations in concepts like "LLM" and "vulnerability detection" may lead to oversights in initial searches. To mitigate these risks, we implemented a systematic approach. We began by analyzing published papers from established conferences and journals. We extracted core keywords from these sources. Over a two-month period, we refined our selection from approximately 500 papers to 58 highly relevant studies. Future versions of this survey will incorporate new developments in this rapidly evolving field. This ongoing process will ensure more comprehensive and timely coverage of research literature.

5 Conclusion

This study presents a systematic analysis of LLM applications in vulnerability detection. Through extensive literature review, we provide a comprehensive examination of the current research landscape, systematically addressing four key questions: the application of LLMs in vulnerability detection, the design of evaluation benchmarks and datasets, current technical approaches, and existing challenges with future directions.

Our findings demonstrate that LLMs exhibit significant potential in code comprehension and vulnerability detection. Through techniques such as fine-tuning and prompt engineering, LLMs can effectively improve detection accuracy. Experiments across multiple benchmark datasets indicate that recent large-scale LLMs, such as GPT-4 and Claude-3.5, have achieved notable progress in vulnerability detection tasks. However, significant challenges remain in applying LLMs to practical security development. The primary obstacle is the scarcity of high-quality datasets, which constrains model training and evaluation. Additionally, current LLMs show notable limitations in handling complex code structures and repository-level vulnerability detection. Furthermore, issues regarding output randomness and model explainability require further investigation.

Based on these findings, we propose several promising research directions: enhancing model adaptation to code evolution, improving vulnerability reproduction and repair capabilities, developing high-quality datasets, and strengthening model robustness and explainability. Advances in these areas will drive the broader adoption of LLMs in vulnerability detection.

In the future, we plan to enrich this review by adding more vulnerability-related tasks, such as vulnerability localization, vulnerability assessment and vulnerability patching.

References

[1] Wasi Uddin Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2021. Unified pre-training for program understanding and generation. *arXiv preprint arXiv:2103.06333* (2021).

[2] Md Tauseef Alam, Raju Halder, and Abyayananda Maiti. 2024. Detection made easy: Potentials of large language models for solidity vulnerabilities. *arXiv preprint arXiv:2409.10574* (2024).

[3] Virendra Ashiwal, Soeren Finster, and Abdallah Dawoud. 2024. LLM-based vulnerability sourcing from unstructured data. In *2024 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. IEEE, 634–641.

[4] Ömer Aslan, Semih Serkant Aktuğ, Merve Ozkan-Okay, Abdullah Asim Yilmaz, and Erdal Akin. 2023. A comprehensive review of cyber security vulnerabilities, threats, attacks, and solutions. *Electronics* 12, 6 (2023).

[5] Syafiq Al Atiiq, Christian Gehrmann, Kevin Dahlén, and Karim Khalil. 2024. From generalist to specialist: Exploring cwe-specific vulnerability detection. *arXiv preprint arXiv:2408.02329* (2024).

[6] Somnath Banerjee, Sayan Layek, Rima Hazra, and Animesh Mukherjee. 2025. How (un) ethical are instruction-centric responses of llms? Unveiling the vulnerabilities of safety guardrails to harmful queries. In *Proceedings of the International AAAI Conference on Web and Social Media*, Vol. 19. 193–205.

[7] Guru Bhandari, Amara Naseer, and Leon Moonen. 2021. CVEfixes: Automated collection of vulnerabilities and their fixes from open-source software. In *Proceedings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering* (Athens, Greece) *(PROMISE 2021)*. Association for Computing Machinery, New York, NY, USA, 30–39.

[8] Biagio Boi, Christian Esposito, and Sokjoon Lee. 2024. Smart contract vulnerability detection: The role of large language model (LLM). *ACM SIGAPP Applied Computing Review* 24, 2 (2024), 19–29.

[9] Biagio Boi, Christian Esposito, and Sokjoon Lee. 2024. VulnHunt-GPT: A smart contract vulnerabilities detector based on OpenAI chatGPT. In *Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing (SAC '24)*. Association for Computing Machinery, New York, NY, USA, 1517–1524.

[10] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*. H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 1877–1901.

[11] Quang-Cuong Bui, Riccardo Scandariato, and Nicolás E. Díaz Ferreyra. 2022. Vul4J: A dataset of reproducible Java vulnerabilities geared towards the study of program repair techniques. In *Proceedings of the 19th International Conference on Mining Software Repositories* (Pittsburgh, Pennsylvania) *(MSR '22)*. Association for Computing Machinery, New York, NY, USA, 464–468.

[12] Marcel Böhme, Ezekiel Olamide Soremekun, Sudipta Chattopadhyay, Emamurho Juliet Ugherughe, and Andreas Zeller. 2017. How developers debug software - The DBGBENCH dataset. In *2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C)*. 244–246.

[13] Daipeng Cao and W. Jun. 2024. LLM-cloudsec: Large language model empowered automatic and deep vulnerability analysis for intelligent clouds. In *IEEE INFOCOM 2024-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*. IEEE, 1–6.

[14] Di Cao, Yong Liao, and Xiuwei Shang. 2024. RealVul: Can we detect vulnerabilities in web applications with LLM? *arXiv preprint arXiv:2410.07573* (2024).

[15] Saikat Chakraborty, Toufique Ahmed, Yangruibo Ding, Premkumar T. Devanbu, and Baishakhi Ray. 2022. NatGen: Generative pre-training by "naturalizing" source code. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Singapore, Singapore) *(ESEC/FSE 2022)*. Association for Computing Machinery, New York, NY, USA, 18–30.

[16] Saikat Chakraborty, Rahul Krishna, Yangruibo Ding, and Baishakhi Ray. 2022. Deep learning based vulnerability detection: Are we there yet? *IEEE Transactions on Software Engineering* 48, 9 (2022), 3280–3296.

[17] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).

[18] Yizheng Chen, Zhoujie Ding, Lamya Alowain, Xinyun Chen, and David Wagner. 2023. DiverseVul: A new vulnerable source code dataset for deep learning based vulnerability detection. In *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses* (Hong Kong, China) *(RAID '23)*. Association for Computing Machinery, New York, NY, USA, 654–668.

[19] Ying-Jung Chen and Vijay K Madisetti. 2024. Information security, ethics, and integrity in llm agent interaction. *Journal of Information Security* 16, 1 (2024), 184–196.

[20] Brian Chess and Gary McGraw. 2004. Static analysis for security. *IEEE Security & Privacy* 2, 6 (2004), 76–79.

[21] Min-Je Choi, Sehun Jeong, Hakjoo Oh, and Jaegul Choo. 2017. End-to-end prediction of buffer overruns from raw source code via neural memory networks. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence* (Melbourne, Australia) *(IJCAI'17)*. AAAI Press, 1546–1553.

[22] CVE Numbering Authorities (CNAs). 2024. Retrieved from https://www.cve.org/programorganization/cnas

[23] Haixing Dai, Zhengliang Liu, Wenxiong Liao, Xiaoke Huang, Yihan Cao, Zihao Wu, Lin Zhao, Shaochen Xu, Wei Liu, Ninghao Liu, et al. 2023. Auggpt: Leveraging chatgpt for text data augmentation. *arXiv preprint arXiv:2302.13007* (2023).

[24] DARPA & ARPA-H AI Cyber Challenge Initiative. 2025. AIxCC Competition Archive. Retrieved from https://archive.aicyberchallenge.com/. Accessed: 2025-08-22.

[25] DARPA and ARPA-H. 2024. Artificial Intelligence Cyber Challenge (AIxCC). Retrieved from https://aicyberchallenge.com/ [Accessed: 01-24-2025].

[26] Defense Advanced Research Projects Agency (DARPA), Information Innovation Office. 2025. *AIxCC Final Competition Procedures and Scoring Guide, Version 2*. Technical Report. DARPA. Retrieved from https://aicyberchallenge.com/final-competition-procedures-and-scoring-guide/ Release Date: June 6, 2025; Downloaded from AIxCC website.

[27] Xiao Deng, Fuyao Duan, Rui Xie, Wei Ye, and Shikun Zhang. 2024. Improving long-tail vulnerability detection through data augmentation based on large language models. In *2024 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 262–274.

[28] Hao Ding, Yizhou Liu, Xuefeng Piao, Huihui Song, and Zhenzhou Ji. 2025. SmartGuard: An LLM-enhanced framework for smart contract vulnerability detection. *Expert Systems with Applications* 269 (2025), 126479. DOI: https://doi.org/10.1016/j.eswa.2025.126479

[29] Yangruibo Ding, Yanjun Fu, Omniyyah Ibrahim, Chawin Sitawarin, Xinyun Chen, Basel Alomair, David Wagner, Baishakhi Ray, and Yizheng Chen. 2025. Vulnerability detection with code language models: How far are we? . In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, Los Alamitos, CA, USA, 469–481. DOI: https://doi.org/10.1109/ICSE55347.2025.00038

[30] Xiaohu Du, Ming Wen, Jiahao Zhu, Zifan Xie, Bin Ji, Huijun Liu, Xuanhua Shi, and Hai Jin. 2024. Generalization-enhanced code vulnerability detection via multi-task instruction fine-tuning. *arXiv preprint arXiv:2406.03718* (2024).

[31] Xueying Du, Geng Zheng, Kaixin Wang, Jiayi Feng, Wentai Deng, Mingwei Liu, Bihuan Chen, Xin Peng, Tao Ma, and Yiling Lou. 2024. Vul-RAG: Enhancing LLM-based vulnerability detection via knowledge-level RAG. *arXiv preprint arXiv:2406.11147* (2024).

[32] Adanma Cecilia Eberendu, Valentine Ikechukwu Udegbe, Edmond Onwubiko Ezennorom, Anita Chinonso Ibegbulam, Titus Ifeanyi Chinebu, et al. 2022. A systematic literature review of software vulnerability detection. *European Journal of Computer Science and Information Technology* 10, 1 (2022), 23–37.

[33] Jiahao Fan, Yi Li, Shaohua Wang, and Tien N. Nguyen. 2020. A C/C++ code vulnerability dataset with code changes and CVE summaries. In *Proceedings of the 17th International Conference on Mining Software Repositories* (Seoul, Republic of Korea) *(MSR '20)*. Association for Computing Machinery, New York, NY, USA, 508–512.

[34] Richard Fang, Rohan Bindu, Akul Gupta, and Daniel Kang. 2024. Llm agents can autonomously exploit one-day vulnerabilities. *arXiv preprint arXiv:2404.08144* (2024).

[35] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A pre-trained model for programming and natural languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. Trevor Cohn, Yulan He, and Yang Liu (Eds.). Association for Computational Linguistics, Online, 1536–1547.

[36] Mohamed Amine Ferrag, Ammar Battah, Norbert Tihanyi, Ridhi Jain, Diana Maimuţ, Fatima Alwahedi, Thierry Lestable, Narinderjit Singh Thandi, Abdechakour Mechri, Merouane Debbah, et al. 2025. Securefalcon: Are we there yet in automated software vulnerability detection with llms? *IEEE Transactions on Software Engineering* 51, 4 (2025), 1248–1265.

[37] João F. Ferreira, Pedro Cruz, Thomas Durieux, and Rui Abreu. 2021. SmartBugs: A framework to analyze solidity smart contracts. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering* (Virtual Event, Australia) *(ASE '20)*. Association for Computing Machinery, New York, NY, USA, 1349–1352.

[38] NSA Center for Assured Software. 2024. Software Assurance Reference Dataset (SARD): Juliet C/C++ 1.3. Retrieved from https://samate.nist.gov/SARD/test-suites/112 Accessed: November 10, 2024.

[39] NSA Center for Assured Software. 2024. Software Assurance Reference Dataset (SARD): Juliet Java 1.3. Retrieved from https://samate.nist.gov/SARD/test-suites/111 Accessed: November 10, 2024.

[40] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen-tau Yih, Luke Zettlemoyer, and Mike Lewis. 2022. Incoder: A generative model for code infilling and synthesis. *arXiv preprint arXiv:2204.05999* (2022).

[41] Michael Fu, Chakkrit Kla Tantithamthavorn, Van Nguyen, and Trung Le. 2023. Chatgpt for vulnerability detection, classification, and repair: How far are we?. In *2023 30th Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, 632–636.

[42] Yuyou Gan, Yong Yang, Zhe Ma, Ping He, Rui Zeng, Yiming Wang, Qingming Li, Chunyi Zhou, Songze Li, Ting Wang, et al. 2024. Navigating the risks: A survey of security, privacy, and ethics threats in llm-based agents. *arXiv preprint arXiv:2411.09523* (2024).

[43] Xiang Gao, Bo Wang, Gregory J. Duck, Ruyi Ji, Yingfei Xiong, and Abhik Roychoudhury. 2021. Beyond tests: Program vulnerability repair via crash constraint extraction. *ACM Trans. Softw. Eng. Methodol.* 30, 2, Article 14 (Feb. 2021), 27 pages.

[44] Zeyu Gao, Hao Wang, Yuchen Zhou, Wenyu Zhu, and Chao Zhang. 2023. How far have we gone in vulnerability detection using large language models. *arXiv preprint arXiv:2311.12420* (2023).

[45] Asem Ghaleb and Karthik Pattabiraman. 2020. How effective are smart contract analysis tools? Evaluating smart contract static analysis tools using bug injection. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis* (Virtual Event, USA) *(ISSTA 2020)*. Association for Computing Machinery, New York, NY, USA, 415–427.

[46] Rikhiya Ghosh, Oladimeji Farri, Hans-Martin von Stockhausen, Martin Schmitt, and George Marica Vasile. 2024. CVE-LLM: Automatic vulnerability evaluation in medical device industry using large language models. *arXiv preprint arXiv:2407.14640* (2024).

[47] GitHub. 2023. GitHub Copilot. Retrieved from https://github.com/features/copilot Accessed: 2024-11-14.
[48] José Gonçalves, Tiago Dias, Eva Maia, and Isabel Praça. 2024. Scope: Evaluating llms for software vulnerability detection. *arXiv preprint arXiv:2407.14372* (2024).
[49] Google. 2025. LLM-powered fuzzing via OSS-Fuzz. Retrieved from https://github.com/google/oss-fuzz-gen/tree/main. Apache-2.0 license, 1.3k stars, 192 forks, active development.
[50] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2022. UniXcoder: Unified cross-modal pre-training for code representation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (Eds.). Association for Computational Linguistics, Dublin, Ireland, 7212–7225.
[51] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, et al. 2020. Graphcodebert: Pre-training code representations with data flow. *arXiv preprint arXiv:2009.08366* (2020).
[52] Yuejun Guo, Constantinos Patsakis, Qiang Hu, Qiang Tang, and Fran Casino. 2024. Outside the comfort zone: Analysing LLM capabilities in software vulnerability detection. In *Computer Security – ESORICS 2024*, Joaquin Garcia-Alfaro, Rafał Kozik, Michał Choraś, and Sokratis Katsikas (Eds.). Vol. 14982. Springer Nature Switzerland, Cham, 271–289. Series Title: Lecture Notes in Computer Science.
[53] Hazim Hanif and Sergio Maffeis. 2022. VulBERTa: Simplified source code pre-training for vulnerability detection. In *2022 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 1–8.
[54] Jean Haurogné, Nihala Basheer, and Shareeful Islam. [n. d.]. Advanced vulnerability detection using llm with transparency obligation practice towards trustworthy ai. *Available at SSRN 4925500* ([n. d.]).
[55] Jean Haurogné, Nihala Basheer, and Shareeful Islam. 2024. Vulnerability detection using BERT based LLM model with transparency obligation practice towards trustworthy AI. *Machine Learning with Applications* 18, 1 (2024), 100598.
[56] Jean Haurogné, Nihala Basheer, and Shareeful Islam. 2024. Vulnerability detection using BERT based LLM model with transparency obligation practice towards trustworthy AI. *Machine Learning with Applications* 18, 1 (2024), 100598.
[57] Ahmad Hazimeh, Adrian Herrera, and Mathias Payer. 2020. Magma: A ground-truth fuzzing benchmark. *Proc. ACM Meas. Anal. Comput. Syst.* 4, 3, Article 49 (Nov. 2020), 29 pages.
[58] Jingxuan He and Martin Vechev. 2023. Large language models for code: Security hardening and adversarial testing. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security* (Copenhagen, Denmark) *(CCS '23)*. Association for Computing Machinery, New York, NY, USA, 1865–1879.
[59] Junda He, Xin Zhou, Bowen Xu, Ting Zhang, Kisub Kim, Zhou Yang, Ferdian Thung, Ivana Clairine Irsan, and David Lo. 2024. Representation learning for stack overflow posts: How far are we? *ACM Transactions on Software Engineering and Methodology* 33, 3 (2024), 1–24.
[60] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large language models for software engineering: A systematic literature review. *ACM Trans. Softw. Eng. Methodol.* 33, 8 (Sept. 2024), 79.
[61] Sihao Hu, Tiansheng Huang, Fatih İlhan, Selim Furkan Tekin, and Ling Liu. 2023. Large language model-powered smart contract vulnerability detection: New perspectives. In *2023 5th IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA)*. 297–306.
[62] Mahmoud Jahanshahi and Audris Mockus. 2025. Cracks in the stack: Hidden vulnerabilities and licensing risks in llm pre-training datasets. In *2025 IEEE/ACM International Workshop on Large Language Models for Code (LLM4Code)*. IEEE, 104–111.
[63] Yu Jiang, Jie Liang, Fuchen Ma, Yuanliang Chen, Chijin Zhou, Yuheng Shen, Zhiyong Wu, Jingzhou Fu, Mingzhe Wang, Shanshan Li, and Quan Zhang. 2024. When fuzzing meets LLMs: Challenges and opportunities. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE 2024)*. Association for Computing Machinery, New York, NY, USA, 492–496.
[64] Brittany Johnson, Yoonki Song, Emerson Murphy-Hill, and Robert Bowdidge. 2013. Why don't software developers use static analysis tools to find bugs?. In *2013 35th International Conference on Software Engineering (ICSE)*. 672–681.
[65] Aditya Kanade, Petros Maniatis, Gogul Balakrishnan, and Kensen Shi. 2020. Learning and evaluating contextual embedding of source code. In *Proceedings of the 37th International Conference on Machine Learning (ICML'20)*. JMLR.org, Article 474, 12 pages.
[66] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361* (2020).
[67] Wafaa Kasri, Yassine Himeur, Hamzah Ali Alkhazaleh, Saed Tarapiah, Shadi Atalla, Wathiq Mansoor, and Hussain Al-Ahmad. 2025. From vulnerability to defense: The role of large language models in enhancing cybersecurity. *Computation* 13, 2 (2025), 30.

[68] Mete Keltek, Rong Hu, Mohammadreza Fani Sani, and Ziyue Li. 2024. LSAST–enhancing cybersecurity through LLM-supported static application security testing. *arXiv preprint arXiv:2409.15735* (2024).
[69] Avishree Khare, Saikat Dutta, Ziyang Li, Alaia Solko-Breslin, Rajeev Alur, and Mayur Naik. 2023. Understanding the effectiveness of large language models in detecting security vulnerabilities. *arXiv preprint arXiv:2311.16169* (2023).
[70] Vasileios Kouliaridis, Georgios Karopoulos, and Georgios Kambourakis. 2024. Assessing the effectiveness of LLMs in android application vulnerability analysis. *arXiv preprint arXiv:2406.18894* (2024).
[71] Ummay Kulsum, Haotian Zhu, Bowen Xu, and Marcelo d'Amorim. 2024. A case study of llm for automated vulnerability repair: Assessing impact of reasoning and patch validation feedback. In *Proceedings of the 1st ACM International Conference on AI-Powered Software*. 103–111.
[72] Chris Lattner and Vikram Adve. 2004. LLVM: A compilation framework for lifelong program analysis & transformation. In *International Symposium on Code Generation and Optimization, 2004. CGO 2004.* IEEE, 75–86.
[73] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems* 33 (2020), 9459–9474.
[74] Haonan Li, Yu Hao, Yizhuo Zhai, and Zhiyun Qian. 2024. Enhancing static analysis for practical bug detection: An LLM-integrated approach. *Proceedings of the ACM on Programming Languages* 8, OOPSLA1 (2024), 474–499.
[75] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. 2023. Starcoder: May the source be with you! *arXiv preprint arXiv:2305.06161* (2023).
[76] Tsz-On Li, Wenxi Zong, Yibo Wang, Haoye Tian, Ying Wang, Shing-Chi Cheung, and Jeff Kramer. 2023. Nuances are the key: Unlocking chatgpt to find failure-inducing tests with differential prompting. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 14–26.
[77] Ziyang Li, Saikat Dutta, and Mayur Naik. 2024. LLM-assisted static analysis for detecting security vulnerabilities. *arXiv preprint arXiv:2405.17238* (2024).
[78] Zhen Li, Deqing Zou, Shouhuai Xu, Zhaoxuan Chen, Yawei Zhu, and Hai Jin. 2021. Vuldeelocator: A deep learning-based fine-grained vulnerability detector. *IEEE Transactions on Dependable and Secure Computing* 19, 4 (2021), 2821–2837.
[79] Zhen Li, Deqing Zou, Shouhuai Xu, Hai Jin, Yawei Zhu, and Zhaoxuan Chen. 2021. Sysevr: A framework for using deep learning to detect software vulnerabilities. *IEEE Transactions on Dependable and Secure Computing* 19, 4 (2021), 2244–2258.
[80] Zhen Li, Deqing Zou, Shouhuai Xu, Xinyu Ou, Hai Jin, Sujuan Wang, Zhijun Deng, and Yuyi Zhong. 2018. Vuldeepecker: A deep learning-based system for vulnerability detection. *arXiv preprint arXiv:1801.01681* (2018).
[81] Guanjun Lin, Jun Zhang, Wei Luo, Lei Pan, and Yang Xiang. 2017. POSTER: Vulnerability discovery with function representation learning from unlabeled projects. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. 2539–2541.
[82] Fengyu Liu, Yuan Zhang, Jiaqi Luo, Jiarun Dai, Tian Chen, Letian Yuan, Zhengmin Yu, Youkun Shi, Ke Li, Chengyuan Zhou, and Min Yang. 2025. Make agent defeat agent: Automatic detection of taint-style vulnerabilities in LLM-based agents. In *Proceedings of the 34th USENIX Security Symposium (USENIX Security 2025)*. USENIX Association, Seattle, WA, USA.
[83] Shangqing Liu, Wei Ma, Jian Wang, Xiaofei Xie, Ruitao Feng, and Yang Liu. 2024. Enhancing code vulnerability detection via vulnerability-preserving data augmentation. In *Proceedings of the 25th ACM SIGPLAN/SIGBED International Conference on Languages, Compilers, and Tools for Embedded Systems*. 166–177.
[84] Xiaohao Liu, Jie Wu, Zhulin Tao, Yunshan Ma, Yinwei Wei, and Tat-seng Chua. 2024. Harnessing large language models for multimodal product bundling. *arXiv preprint arXiv:2407.11712* (2024).
[85] Yu Liu, Lang Gao, Mingxin Yang, Yu Xie, Ping Chen, Xiaojin Zhang, and Wei Chen. 2024. VulDetectBench: Evaluating the deep capability of vulnerability detection with large language models. *arXiv preprint arXiv:2406.07595* (2024).
[86] Zhenguang Liu, Peng Qian, Jiaxu Yang, Lingfeng Liu, Xiaojun Xu, Qinming He, and Xiaosong Zhang. 2023. Rethinking smart contract fuzzing: Fuzzing with invocation ordering and important branch revisiting. *IEEE Transactions on Information Forensics and Security* 18, 4 (2023), 1237–1251.
[87] Zhihong Liu, Zezhou Yang, and Qing Liao. 2024. Exploration on prompting LLM with code-specific information for vulnerability detection. In *2024 IEEE International Conference on Software Services Engineering (SSE)*. IEEE, 273–281.
[88] V Benjamin Livshits and Monica S Lam. 2005. Finding security vulnerabilities in Java applications with static analysis.. In *USENIX Security Symposium*, Vol. 14. 18–18.
[89] Guilong Lu, Xiaolin Ju, Xiang Chen, Wenlong Pei, and Zhilong Cai. 2024. GRACE: Empowering LLM-based software vulnerability detection with graph structure and in-context learning. *Journal of Systems and Software* 212, C (June 2024), 112031.

[90] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, et al. 2021. Codexglue: A machine learning benchmark dataset for code understanding and generation. *arXiv preprint arXiv:2102.04664* (2021).

[91] Yu Luo, Weifeng Xu, Karl Andersson, Mohammad Shahadat Hossain, and Dianxiang Xu. 2024. FELLMVP: An ensemble LLM framework for classifying smart contract vulnerabilities. In *2024 IEEE International Conference on Blockchain (Blockchain)*. IEEE, 89–96.

[92] Yunlong Lyu, Yuxuan Xie, Peng Chen, and Hao Chen. 2024. Prompt fuzzing for fuzz driver generation. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 3793–3807.

[93] Wei Ma, Daoyuan Wu, Yuqiang Sun, Tianwen Wang, Shangqing Liu, Jian Zhang, Yue Xue, and Yang Liu. 2024. Combining fine-tuning and LLM-based agents for intuitive smart contract auditing with justifications. *arXiv preprint arXiv:2403.16073* (2024).

[94] Andrew A Mahyari. 2024. Harnessing the power of LLMs in source code vulnerability detection. In *MILCOM 2024-2024 IEEE Military Communications Conference (MILCOM)*. IEEE, 251–256.

[95] Qiheng Mao, Zhenhao Li, Xing Hu, Kui Liu, Xin Xia, and Jianling Sun. 2024. Towards effectively detecting and explaining vulnerabilities using large language models. *arXiv preprint arXiv:2406.09701* (2024).

[96] Zhenyy Mao, Jialong Li, Dongming Jin, Munan Li, and Kenji Tei. 2024. Multi-role consensus through llms discussions for vulnerability detection. In *2024 IEEE 24th International Conference on Software Quality, Reliability, and Security Companion (QRS-C)*. IEEE, 1318–1319.

[97] Florence Martin, Yan Chen, Robert L Moore, and Carl D Westine. 2020. Systematic review of adaptive learning research designs, context, strategies, and technologies from 2009 to 2018. *Educational Technology Research and Development* 68, 4 (2020), 1903–1929.

[98] Noble Saji Mathews, Yelizaveta Brus, Yousra Aafer, Meiyappan Nagappan, and Shane McIntosh. 2024. Llbezpeky: Leveraging large language models for vulnerability detection. *arXiv preprint arXiv:2401.01269* (2024).

[99] Joydeep Mitra and Venkatesh-Prasad Ranganath. 2017. Ghera: A repository of android app vulnerability benchmarks. In *Proceedings of the 13th International Conference on Predictive Models and Data Analytics in Software Engineering*. 43–52.

[100] Maximilian Mozes, Xuanli He, Bennett Kleinberg, and Lewis D Griffin. 2023. Use of llms for illicit purposes: Threats, prevention measures, and vulnerabilities. *arXiv preprint arXiv:2308.12833* (2023).

[101] National Institute of Standards and Technology. 2024. Software Assurance Reference Dataset (SARD). Retrieved from https://samate.nist.gov/SARD/ Accessed: 2024-11-10.

[102] Xu Nie, Ningke Li, Kailong Wang, Shangguang Wang, Xiapu Luo, and Haoyu Wang. 2023. Understanding and tackling label errors in deep learning-based vulnerability detection (experience paper). In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 52–63.

[103] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2022. Codegen: An open large language model for code with multi-turn program synthesis. *arXiv preprint arXiv:2203.13474* (2022).

[104] o2lab. 2025. afc-crs-all-you-need-is-a-fuzzing-brain. Retrieved from https://github.com/o2lab/afc-crs-all-you-need-is-a-fuzzing-brain. GitHub repository, accessed August 22, 2025.

[105] OpenAI. 2022. GPT-3.5. Retrieved from https://platform.openai.com/docs/models Accessed: 2024-11-14.

[106] OpenAI. 2023. *GPT-4 Technical Report*. Technical Report. OpenAI.

[107] Shengyi Pan, Lingfeng Bao, Xin Xia, David Lo, and Shanping Li. 2023. Fine-grained commit-level vulnerability type prediction by CWE tree structure. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 957–969.

[108] Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. 2024. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering* 36, 7 (2024), 3580–3599.

[109] Serena Elisa Ponta, Henrik Plate, Antonino Sabetta, Michele Bezzi, and Cédric Dangremont. 2019. A manually-curated dataset of fixes to vulnerabilities of open-source software. In *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*. IEEE, 383–387.

[110] Moumita Das Purba, Arpita Ghosh, Benjamin J Radford, and Bill Chu. 2023. Software vulnerability detection using large language models. In *2023 IEEE 34th International Symposium on Software Reliability Engineering Workshops (ISSREW)*. IEEE, 112–119.

[111] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI Blog* 1, 8 (2019), 9.

[112] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research* 21, 140 (2020), 1–67.

[113] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950* (2023).

[114] Alejandro Russo and Andrei Sabelfeld. 2010. Dynamic vs. static flow-sensitive security analysis. In *2010 23rd IEEE Computer Security Foundations Symposium*. IEEE, 186–199.

[115] Seyed Mohammad Taghavi Far and Farid Feyzi. 2025. Large language models for software vulnerability detection: A guide for researchers on models, methods, techniques, datasets, and metrics. *International Journal of Information Security* 24, 2 (2025), 78–107. https://doi.org/10.1007/s10207-025-00992-7

[116] Md Nazmus Sakib, Md Athikul Islam, Royal Pathak, and Md Mashrur Arifin. 2024. Risks, causes, and mitigations of widespread deployments of large language models (llms): A survey. In *2024 2nd International Conference on Artificial Intelligence, Blockchain, and Internet of Things (AIBThings)*. IEEE, 1–7.

[117] Kosta Serebryany. 2016. Continuous fuzzing with libFuzzer and AddressSanitizer. In *2016 IEEE Cybersecurity Development (SecDev)*. 157–157.

[118] Alexey Shestov, Anton Cheshkov, Rodion Levichev, Ravil Mussabayev, Pavel Zadorozhny, Evgeny Maslov, Chibirev Vadim, and Egor Bulychev. 2024. Finetuning large language models for vulnerability detection. *arXiv preprint arXiv:2401.17010* (2024).

[119] Parul V. Sindhwad, Prateek Ranka, Siddhi Muni, and Faruk Kazi. 2024. VulnArmor: Mitigating software vulnerabilities with code resolution and detection techniques. *International Journal of Information Technology* 16, 4 (March 2024), 1765–1780.

[120] Sunbeom So and Hakjoo Oh. 2023. SmartFix: Fixing vulnerable smart contracts by accelerating generate-and-verify repair using statistical models. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (San Francisco, CA, USA) *(ESEC/FSE 2023)*. Association for Computing Machinery, New York, NY, USA, 185–197.

[121] Yuqiang Sun, Daoyuan Wu, Yue Xue, Han Liu, Wei Ma, Lyuye Zhang, Yang Liu, and Yingjiu Li. 2024. Llm4vuln: A unified evaluation framework for decoupling and enhancing llms' vulnerability reasoning. *arXiv preprint arXiv:2401.16185* (2024).

[122] Jeniya Tabassum, Mounica Maddela, Wei Xu, and Alan Ritter. 2020. Code and named entity recognition in stackoverflow. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault (Eds.). Association for Computational Linguistics, Online, 4913–4926.

[123] Masoud Jamshidiyan Tehrani and Sattar Hashemi. 2024. Assessing vulnerability in smart contracts: The role of code complexity metrics in security analysis. *arXiv preprint arXiv:2411.17343* (2024).

[124] trailofbits. 2025. afc-buttercup: Buttercup CRS as submitted to the AIxCC Final Competition. Retrieved from https://github.com/trailofbits/afc-buttercup. GitHub repository (archived on August 7, 2025), accessed August 22, 2025.

[125] Saad Ullah, Mingji Han, Saurabh Pujar, Hammond Pearce, Ayse Coskun, and Gianluca Stringhini. 2024. LLMs cannot reliably identify and reason about security vulnerabilities (yet?): A comprehensive evaluation, framework, and benchmarks. In *IEEE Symposium on Security and Privacy*.

[126] U.S. Government Accountability Office (GAO). 2024. CrowdStrike Chaos Highlights Key Cyber Vulnerabilities with Software Updates. [Accessed: 10-24-2024].

[127] A Vaswani. 2017. Attention is all you need. *Advances in Neural Information Processing Systems* (2017), 6000–6010.

[128] Radhika D. Venkatasubramanyam and Sowmya G. R. 2014. Why is dynamic analysis not used as extensively as static analysis: An industrial study. In *Proceedings of the 1st International Workshop on Software Engineering Research and Industrial Practices* (Hyderabad, India) *(SER&IPs 2014)*. Association for Computing Machinery, New York, NY, USA, 24–33.

[129] Inacio Vieira, Will Allred, Séamus Lankford, Sheila Castilho, and Andy Way. 2024. How much data is enough data? Fine-tuning large language models for in-house translation: Performance evaluation across multiple dataset sizes. *arXiv preprint arXiv:2409.03454* (2024).

[130] Jin Wang, Zishan Huang, Hengli Liu, Nianyi Yang, and Yinhao Xiao. 2023. Defecthunter: A novel llm-driven boosted-conformer-based code vulnerability detection mechanism. *arXiv preprint arXiv:2309.15324* (2023).

[131] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. 2024. A survey on large language model based autonomous agents. *Frontiers of Computer Science* 18, 6 (2024), 186345.

[132] Shichao Wang, Yun Zhang, Liagfeng Bao, Xin Xia, and Minghui Wu. 2022. VCMatch: A ranking-based approach for automatic security patches localization for OSS vulnerabilities. In *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 589–600.

[133] Yue Wang, Weishi Wang, Shafiq Joty, and Steven C.H. Hoi. 2021. CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (Eds.). Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 8696–8708.

[134] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems* 35 (2022), 24824–24837.

[135] C.A. Welty. 1997. Augmenting abstract syntax trees for program understanding. In *Proceedings 12th IEEE International Conference Automated Software Engineering*. 126–133.

[136] Xin-Cheng Wen, Cuiyun Gao, Shuzheng Gao, Yang Xiao, and Michael R. Lyu. 2024. SCALE: Constructing structured natural language comment trees for software vulnerability detection. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Vienna, Austria) *(ISSTA 2024)*. Association for Computing Machinery, New York, NY, USA, 235–247.

[137] Qiushi Wu and Kangjie Lu. 2021. On the feasibility of stealthily introducing vulnerabilities in open-source software via hypocrite commits. *Proc. Oakland* 2021 (2021), 17.

[138] Yi Wu, Nan Jiang, Hung Viet Pham, Thibaud Lutellier, Jordan Davis, Lin Tan, Petr Babkin, and Sameena Shah. 2023. How effective are neural networks for fixing security vulnerabilities. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Seattle, WA, USA) *(ISSTA 2023)*. Association for Computing Machinery, New York, NY, USA, 1282–1294.

[139] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. 2024. Fuzz4All: Universal fuzzing with large language models. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (Lisbon, Portugal) *(ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 126, 13 pages.

[140] Frank F. Xu, Uri Alon, Graham Neubig, and Vincent Josua Hellendoorn. 2022. A systematic evaluation of large language models of code. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming* (San Diego, CA, USA) *(MAPS 2022)*. Association for Computing Machinery, New York, NY, USA, 1–10.

[141] HanXiang Xu, ShenAo Wang, Ningke Li, Yanjie Zhao, Kai Chen, Kailong Wang, Yang Liu, Ting Yu, and HaoYu Wang. 2024. Large language models for cyber security: A systematic literature review. *arXiv preprint arXiv:2405.04760* (2024).

[142] Aidan Z. H. Yang, Claire Le Goues, Ruben Martins, and Vincent Hellendoorn. 2024. Large language models for test-free fault localization. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (Lisbon, Portugal) *(ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 17, 12 pages.

[143] Chenyuan Yang, Yinlin Deng, Runyu Lu, Jiayi Yao, Jiawei Liu, Reyhaneh Jabbarvand, and Lingming Zhang. 2024. WhiteFox: White-box compiler fuzzing empowered by large language models. *Proc. ACM Program. Lang.* 8, OOPSLA2, Article 296 (Oct. 2024), 27 pages.

[144] Chenyuan Yang, Zijie Zhao, and Lingming Zhang. 2023. Kernelgpt: Enhanced kernel fuzzing via large language models. *arXiv preprint arXiv:2401.00563* (2023).

[145] Yilin Yang. 2023. Iot software vulnerability detection techniques through large language model. In *International Conference on Formal Engineering Methods*. Springer, 285–290.

[146] Yifan Yao, Jinhao Duan, Kaidi Xu, Yuanfang Cai, Zhibo Sun, and Yue Zhang. 2024. A survey on large language model (LLM) security and privacy: The good, the bad, and the ugly. *High-Confidence Computing* 4, 2 (2024), 100211.

[147] Junjian Ye, Xincheng Fei, Xavier de Carné de Carnavalet, Lianying Zhao, Lifa Wu, and Mengyuan Zhang. 2024. Detecting command injection vulnerabilities in linux-based embedded firmware with LLM-based taint analysis of library functions. *Computers & Security* 144, C (2024), 103971.

[148] Recep Yıldırım, Kerem Aydın, and Orçun Çetin. 2024. Evaluating the impact of conventional code analysis against large language models in API vulnerability detection. In *Proceedings of the 2024 European Interdisciplinary Cybersecurity Conference* (Xanthi, Greece) *(EICC '24)*. Association for Computing Machinery, New York, NY, USA, 57–64.

[149] Xin Yin, Chao Ni, and Shaohua Wang. 2024. Multitask-based evaluation of open-source LLM on software vulnerability. *IEEE Trans. Softw. Eng.* 50, 11 (Nov. 2024), 3071–3087.

[150] Jonas Zaddach, Luca Bruno, Aurelien Francillon, Davide Balzarotti, et al. 2014. AVATAR: A framework to support dynamic security analysis of embedded systems' firmwares. In *NDSS*, Vol. 14. 1–16.

[151] Bing Zhang, Jingyue Li, Jiadong Ren, and Guoyan Huang. 2021. Efficiency and effectiveness of web application vulnerability detection approaches: A review. *ACM Comput. Surv.* 54, 9, Article 190 (Oct. 2021), 35 pages.

[152] Chenhui Zhang, Le Wang, Dunqiu Fan, Junyi Zhu, Tang Zhou, Liyi Zeng, and Zhaohua Li. 2024. VTT-LLM: Advancing vulnerability-to-tactic-and-technique mapping through fine-tuning of large language model. *Mathematics* 12, 9 (2024), 1286.

[153] Cen Zhang, Yaowen Zheng, Mingqiang Bai, Yeting Li, Wei Ma, Xiaofei Xie, Yuekang Li, Limin Sun, and Yang Liu. 2024. How effective are they? Exploring large language model based fuzz driver generation. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Vienna, Austria) *(ISSTA 2024)*. Association for Computing Machinery, New York, NY, USA, 1223–1235.

[154] Jian Zhang, Chong Wang, Anran Li, Weisong Sun, Cen Zhang, Wei Ma, and Yang Liu. 2024. An empirical study of automated vulnerability localization with large language models. *arXiv preprint arXiv:2404.00287* (2024).

[155] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. *arXiv preprint arXiv:2303.18223* (2023).

[156] Yunhui Zheng, Saurabh Pujar, Burn Lewis, Luca Buratti, Edward Epstein, Bo Yang, Jim A. Laredo, Alessandro Morari, and Zhong Su. 2021. D2A: A dataset built for AI-based vulnerability detection methods using differential analysis. *arXiv preprint arXiv:2102.07995* (2021).

[157] Xin Zhou, Duc-Manh Tran, Thanh Le-Cong, Ting Zhang, Ivana Clairine Irsan, Joshua Sumarlin, Bach Le, and David Lo. 2024. Comparison of static application security testing tools and large language models for repo-level vulnerability detection. *arXiv preprint arXiv:2407.16235* (2024).

[158] Xin Zhou, Bowen Xu, DongGyun Han, Zhou Yang, Junda He, and David Lo. 2023. CCBERT: Self-supervised code change representation learning. In *2023 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 182–193.

[159] Xin Zhou, Ting Zhang, and David Lo. 2024. Large language model for vulnerability detection: Emerging results and future directions. In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results* (Lisbon, Portugal) *(ICSE-NIER'24)*. Association for Computing Machinery, New York, NY, USA, 47–51.

[160] Xin Zhou, Ting Zhang, and David Lo. 2024. Large language model for vulnerability detection: Emerging results and future directions. In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results* (Lisbon, Portugal) *(ICSE-NIER'24)*. Association for Computing Machinery, New York, NY, USA, 47–51.

[161] Yaqin Zhou, Shangqing Liu, Jingkai Siow, Xiaoning Du, and Yang Liu. 2019. *Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics Via Graph Neural Networks*. Curran Associates Inc., Red Hook, NY, USA.

[162] Yuhui Zhu, Guanjun Lin, Lipeng Song, and Jun Zhang. 2022. The application of neural network for software vulnerability detection: A review. *Neural Comput. Appl.* 35, 2 (Nov. 2022), 1279–1301.

[163] Arastoo Zibaeirad and Marco Vieira. 2024. VulnLLMEval: A framework for evaluating large language models in software vulnerability detection and patching. *arXiv preprint arXiv:2409.10756* (2024).

Received 12 February 2025; revised 25 August 2025; accepted 15 September 2025